



Collections



By: RAJASHEKAR G.S
Java Full Stack Trainer

Introduction to Generics

- **Generics is a term that denotes a set of language features related to the definition and use of Generic types and methods.**
- **Java Generic methods differ from regular data types and methods.**
- **Before Generics, we used the collection to store any type of objects i.e. non-generic.**
- **Now, Generics force the Java programmer to store a specific type of objects.**

Generics

- The Java Generics programming is introduced in J2SE 5 to deal with type-safe objects.
- The idea is to allow type (Integer, String, ... etc and user defined types) to be a parameter to methods, classes and interfaces.
- For example, classes like HashSet, ArrayList, HashMap, etc use generics very well.
- It would be nice if we could write a single sort method that could sort the elements in an Integer array, a String array, or an array of any type that supports ordering.

Generics

- It gives you a provision of telling the compiler the type of the collection.
- So that it can be checked even before execution.
- Once the compiler knows the element type of the collection, the compiler can check for type safety and can insert the correct casts on values being taken out of the collection.

Why Java Generics?

- If you look at the Java collection framework classes, then you will observe that most classes take parameter/argument of type Object.
- Basically, in this form, they can take any Java type as argument and return the same object or argument.
- They are essentially heterogeneous i.e. not of a similar type.
- Sometimes the data type of the input is not fixed. The input can be an integer, a float or Java string.
- When you use Generics, the parameters in the code is checked at compile time automatically and it sets the datatype by default.

Advantage of Java Generics

- **Type-safety:** We can hold only a single type of objects in generics. It doesn't allow to store other objects.
- **Without Generics,** we can store any type of objects.
- **Type casting is not required:** There is no need to typecast the object.
- **Before Generics,** we need to type cast.
- **Compile-Time Checking:** It is checked at compile time so problem will not occur at runtime.
- **The good programming strategy** says it is far better to handle the problem at compile time than runtime.

Types of Java Generics

There are 4 different ways Generics can be applied to in Java and they are as follows:

- **Generic Type Class**
- **Generic Interface**
- **Generic Method**
- **Generic Constructor**

Parameterized Classes and Generics

- The class `ArrayList` is a *parameterized class*
- It has a parameter, denoted by `Base_Type`, that can be replaced by any reference type to obtain a class for `ArrayLists` with the specified base type
- Starting with version 5.0, Java allows class definitions with parameters for types
 - These classes that have type parameters are called *parameterized class* or *generic definitions*, or, simply, *generics*

Generics (Cont'd)

- A class definition with a type parameter is stored in a file and compiled just like any other class.
- Once a parameterized class is compiled, it can be used like any other class.
 - However, the class type plugged in for the type parameter must be specified before it can be used in a program.
 - Doing this is said to *instantiate* the generic class.

Syntax: **BaseType <Type> obj = new BaseType <Type>()**

Sample<String> object = new Sample<String>();

Java Generics Naming Convention

- **By convention, type parameter names are single, uppercase letters.**
- **E - Element (used extensively by the Java Collections Framework)**
- **K - Key**
- **N - Number**
- **T - Type**
- **V - Value**
- **S,U,V etc. - 2nd, 3rd, 4th types**

A Class Definition with a Type Parameter

Display 14.4 A Class Definition with a Type Parameter

```
1  public class Sample<T>
2  {
3      private T data;

4      public void setData(T newData)
5      {
6          data = newData;
7      }

8      public T getData()
9      {
10         return data;
11     }
12 }
```

T is a parameter for a type.

A Class Definition with a Type Parameter (Cont'd)

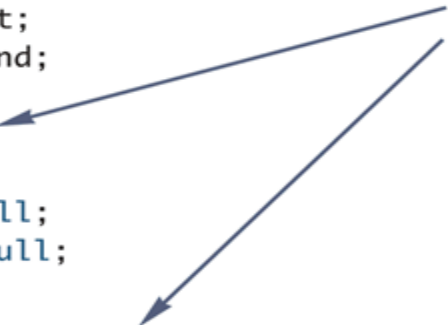
- **A class that is defined with a parameter for a type is called a generic class or a parameterized class**
 - The type parameter is included in angular brackets after the class name in the class definition heading.
 - Any non-keyword identifier can be used for the type parameter, but by convention, the parameter starts with an uppercase letter.
 - The type parameter can be used like other types used in the definition of a class.

Generic Class Definition: An Example

Display 14.5 A Generic Ordered Pair Class

```
1  public class Pair<T>
2  {
3      private T first;
4      private T second;
5
6      public Pair()
7      {
8          first = null;
9          second = null;
10
11      public Pair(T firstItem, T secondItem)
12      {
13          first = firstItem;
14          second = secondItem;
15
16      public void setFirst(T newFirst)
17      {
18          first = newFirst;
19
20      public void setSecond(T newSecond)
21      {
22          second = newSecond;
23
24      public T getFirst()
25      {
26          return first;
```

Constructor headings do not include the type parameter in angular brackets.



(continued)

Generic Class Definition: An Example (Cont'd)

Display 14.5 A Generic Ordered Pair Class

```
27     public T getSecond()
28     {
29         return second;
30     }

31     public String toString()
32     {
33         return ( "first: " + first.toString() + "\n"
34                 + "second: " + second.toString() );
35     }
36
37     public boolean equals(Object otherObject)
38     {
39         if (otherObject == null)
40             return false;
41         else if (getClass() != otherObject.getClass())
42             return false;
43         else
44         {
45             Pair<T> otherPair = (Pair<T>)otherObject;
46             return (first.equals(otherPair.first)
47                     && second.equals(otherPair.second));
48         }
49     }
50 }
```

Generic Class Usage: An Example

Display 14.6 Using Our Ordered Pair Class

```
1  import java.util.Scanner;

2  public class GenericPairDemo
3  {
4      public static void main(String[] args)
5      {
6          Pair<String> secretPair =
7              new Pair<String>("Happy", "Day");
8
9          Scanner keyboard = new Scanner(System.in);
10         System.out.println("Enter two words:");
11         String word1 = keyboard.next();
12         String word2 = keyboard.next();
13         Pair<String> inputPair =
14             new Pair<String>(word1, word2);
15
16         if (inputPair.equals(secretPair))
17         {
18             System.out.println("You guessed the secret words");
19             System.out.println("in the correct order!");
20         }
21         else
22         {
23             System.out.println("You guessed incorrectly.");
24             System.out.println("You guessed");
25             System.out.println(inputPair);
26             System.out.println("The secret words are");
27             System.out.println(secretPair);
28         }
29     }
30 }
```

Generic Class Usage: An Example (Cont'd)

Program Output:

Display 14.6 Using Our Ordered Pair Class

SAMPLE DIALOGUE

Enter two words:

`two words`

You guessed incorrectly.

You guessed

first: `two`

second: `words`

The secret words are

first: `Happy`

second: `Day`

A Generic Constructor Name Has No Type Parameter!!!

- Although the class name in a parameterized class definition has a type parameter attached, the type parameter is not used in the heading of the constructor definition:

```
public Pair<T>()
```

- A constructor can use the type parameter as the type for a parameter of the constructor, but in this case, the angular brackets are not used:

```
public Pair(T first, T second)
```

- However, when a generic class is instantiated, the angular brackets are used:

```
Pair<String> pair = new Pair<String>("Happy", "Day");
```

A Primitive Type Cannot be Plugged in for a Type Parameter!!!

- The type plugged in for a type parameter must always be a reference type:
 - It cannot be a primitive type such as `int`, `double`, or `char`
 - However, now that Java has automatic boxing, this is not a big restriction.
 - Note: Reference types can include arrays.

Limitations on Type Parameter Usage

- Within the definition of a parameterized class definition, there are places where an ordinary class name would be allowed, but a type parameter is not allowed.
- In particular, the type parameter cannot be used in simple expressions using `new` to create a new object
 - For instance, the type parameter cannot be used as a constructor name or like a constructor:

```
T object = new T();  
T[] a = new T[10];
```

Limitations on Generic Class Instantiation

- Arrays such as the following are illegal:

```
Pair<String>[] a =  
    new Pair<String>[10];
```

- Although this is a reasonable thing to want to do, it is not allowed given the way that Java implements generic classes

Using Generic Classes and Automatic Boxing

Display 14.7 Using Our Ordered Pair Class and Automatic Boxing

```
1  import java.util.Scanner;

2  public class GenericPairDemo2
3  {
4      public static void main(String[] args)
5      {
6          Pair<Integer> secretPair =
7              new Pair<Integer>(42, 24);
8
9          Scanner keyboard = new Scanner(System.in);
10         System.out.println("Enter two numbers:");
11         int n1 = keyboard.nextInt();
12         int n2 = keyboard.nextInt();
13         Pair<Integer> inputPair =
14             new Pair<Integer>(n1, n2);
15
16         if (inputPair.equals(secretPair))
17         {
18             System.out.println("You guessed the secret numbers");
19             System.out.println("in the correct order!");
20         }
21         else
22         {
23             System.out.println("You guessed incorrectly.");
24             System.out.println("You guessed");
25             System.out.println(inputPair);
26             System.out.println("The secret numbers are");
27             System.out.println(secretPair);
28         }
29     }
}
```

Automatic boxing allows you to use an `int` argument for an `Integer` parameter.

Using Generic Classes and Automatic Boxing (Cont'd)

Program Output:

Display 14.7 Using Our Ordered Pair Class and Automatic Boxing

SAMPLE DIALOGUE

Enter two numbers:

42 24

You guessed the secret numbers
in the correct order!

Multiple Type Parameters

- **A generic class definition can have any number of type parameters.**
 - Multiple type parameters are listed in angular brackets just as in the single type parameter case, but are separated by commas.

Multiple Type Parameters (Cont'd)

Display 14.8 Multiple Type Parameters

```
1  public class TwoTypePair<T1, T2>
2  {
3      private T1 first;
4      private T2 second;

5      public TwoTypePair()
6      {
7          first = null;
8          second = null;
9      }

10     public TwoTypePair(T1 firstItem, T2 secondItem)
11     {
12         first = firstItem;
13         second = secondItem;
14     }

15     public void setFirst(T1 newFirst)
16     {
17         first = newFirst;
18     }

19     public void setSecond(T2 newSecond)
20     {
21         second = newSecond;
22     }

23     public T1 getFirst()
24     {
25         return first;
26     }
```

(continued)

Multiple Type Parameters (Cont'd)

Display 14.8 Multiple Type Parameters

```
27     public T2 getSecond()
28     {
29         return second;
30     }

31     public String toString()
32     {
33         return ( "first: " + first.toString() + "\n"
34                 + "second: " + second.toString() );
35     }
36
37     public boolean equals(Object otherObject)
38     {
39         if (otherObject == null)
40             return false;
41         else if (getClass() != otherObject.getClass())
42             return false;
43         else
44         {
45             TwoTypePair<T1, T2> otherPair =
46                 (TwoTypePair<T1, T2>)otherObject;
47             return (first.equals(otherPair.first)
48                     && second.equals(otherPair.second));
49         }
50     }
51 }
```

The first equals is the equals of the type T1. The second equals is the equals of the type T2.

A Generic Classes and Exceptions

- It is not permitted to create a generic class with `Exception`, `Error`, `Throwable`, or any descendent class of `Throwable`
 - A generic class cannot be created whose objects are throwable

```
public class GEx<T> extends Exception
```

- The above example will generate a compiler error message

Using a Generic Class with Two Type Parameters

Display 14.9 Using a Generic Class with Two Type Parameters

```
1  import java.util.Scanner;

2  public class TwoTypePairDemo
3  {
4      public static void main(String[] args)
5      {
6          TwoTypePair<String, Integer> rating =
7              new TwoTypePair<String, Integer>("The Car Guys", 8);

8          Scanner keyboard = new Scanner(System.in);
9          System.out.println(
10              "Our current rating for " + rating.getFirst());
11          System.out.println(" is " + rating.getSecond());

12          System.out.println("How would you rate them?");
13          int score = keyboard.nextInt();
14          rating.setSecond(score);
15          System.out.println(
16              "Our new rating for " + rating.getFirst());
17          System.out.println(" is " + rating.getSecond());
18      }
19  }
```

SAMPLE DIALOGUE

```
Our current rating for The Car Guys
is 8
How would you rate them?
10
Our new rating for The Car Guys
is 10
```

Program Output:

Bounds for Type Parameters

- Sometimes it makes sense to restrict the possible types that can be plugged in for a type parameter **T**.

- For instance, to ensure that only classes that implement the **Comparable** interface are plugged in for **T**, define a class as follows:

```
public class RClass<T extends Comparable>
```

- "**extends Comparable**" serves as a *bound* on the type parameter **T**.
- Any attempt to plug in a type for **T** which does not implement the **Comparable** interface will result in a compiler error message.

Bounds for Type Parameters (Cont'd)

- A bound on a type may be a class name (rather than an interface name)
 - Then only descendent classes of the bounding class may be plugged in for the type parameters:

```
public class ExClass<T extends Class1>
```

- A bounds expression may contain multiple interfaces and up to one class.
- If there is more than one type parameter, the syntax is as follows:

```
public class Two<T1 extends Class1, T2 extends Class2 &  
    Comparable>
```

Bounds for Type Parameters (Cont'd)

Display 14.10 A Bounded Type Parameter

```
1  public class Pair<T extends Comparable>
2  {
3      private T first;
4      private T second;

5      public T max()
6      {
7          if (first.compareTo(second) <= 0)
8              return first;
9          else
10             return second;
11     }
```

<All the constructors and methods given in Display 14.5
are also included as part of this generic class definition>

```
12 }
```

Generic Interfaces

- **An interface can have one or more type parameters.**
- **The details and notation are the same as they are for classes with type parameters.**

Generic Methods

- **When a generic class is defined, the type parameter can be used in the definitions of the methods for that generic class.**
- **In addition, a generic method can be defined that has its own type parameter that is not the type parameter of any class**
 - A generic method can be a member of an ordinary class or a member of a generic class that has some other type parameter.
 - The type parameter of a generic method is local to that method, not to the class.

Generic Methods (Cont'd)

- The type parameter must be placed (in angular brackets) after all the modifiers, and before the returned type:

```
public static <T> T genMethod(T[] a)
```

- When one of these generic methods is invoked, the method name is prefaced with the type to be plugged in, enclosed in angular brackets

```
String s = NonG.<String>genMethod(c) ;
```

Inheritance with Generic Classes

- **A generic class can be defined as a derived class of an ordinary class or of another generic class**
 - As in ordinary classes, an object of the subclass type would also be of the superclass type
- **Given two classes: **A** and **B**, and given **G**: a generic class, there is no relationship between **G<A>** and **G****
 - This is true regardless of the relationship between class **A** and **B**, e.g., if class **B** is a subclass of class **A**

A Derived Generic Class: An Example

Display 14.11 A Derived Generic Class

```
1  public class UnorderedPair<T> extends Pair<T>
2  {
3      public UnorderedPair()
4      {
5          setFirst(null);
6          setSecond(null);
7      }
8
9      public UnorderedPair(T firstItem, T secondItem)
10     {
11         setFirst(firstItem);
12         setSecond(secondItem);
13     }
14     public boolean equals(Object otherObject)
15     {
16         if (otherObject == null)
17             return false;
18         else if (getClass() != otherObject.getClass())
19             return false;
20         else
21         {
22             UnorderedPair<T> otherPair =
23                 (UnorderedPair<T>)otherObject;
24             return (getFirst().equals(otherPair.getFirst())
25                 && getSecond().equals(otherPair.getSecond()))
26                 ||
27                 (getFirst().equals(otherPair.getSecond())
28                 && getSecond().equals(otherPair.getFirst()));
29         }
30     }
31 }
```

A Derived Generic Class: An Example (Cont'd)

Display 14.12 Using UnorderedPair

```
1  public class UnorderedPairDemo
2  {
3      public static void main(String[] args)
4      {
5          UnorderedPair<String> p1 =
6              new UnorderedPair<String>("peanuts", "beer");
7          UnorderedPair<String> p2 =
8              new UnorderedPair<String>("beer", "peanuts");
9          if (p1.equals(p2))
10         {
11             System.out.println(p1.getFirst() + " and " +
12                               p1.getSecond() + " is the same as");
13             System.out.println(p2.getFirst() + " and "
14                               + p2.getSecond());
15         }
16     }
17 }
```

Program Output:

SAMPLE DIALOGUE

peanuts and beer is the same as
beer and peanuts

Collection Classes 2-1

- ❑ An object of the Collection class groups multiple elements into a single unit.
- **Collections are used to store, retrieve and manipulate data and to transmit data from one method to another.**
- **Collection Framework is an unified architecture for representing and manipulating collections.**



Collection Classes 2-2

Collection Framework is composed of three components.

Interface

Is abstract data types representing collections

Algorithms

Are methods that perform computations on objects that implement the interface

Implementation

Is actual execution of interfaces

Java collections framework (JCF)

- It is a set of classes and interfaces that implement commonly reusable collection data structures.
- Although referred to as a framework, it works in a manner of a library.
- The JCF provides both interfaces that define various collections and classes that implement them.

Advantages of Collection Framework

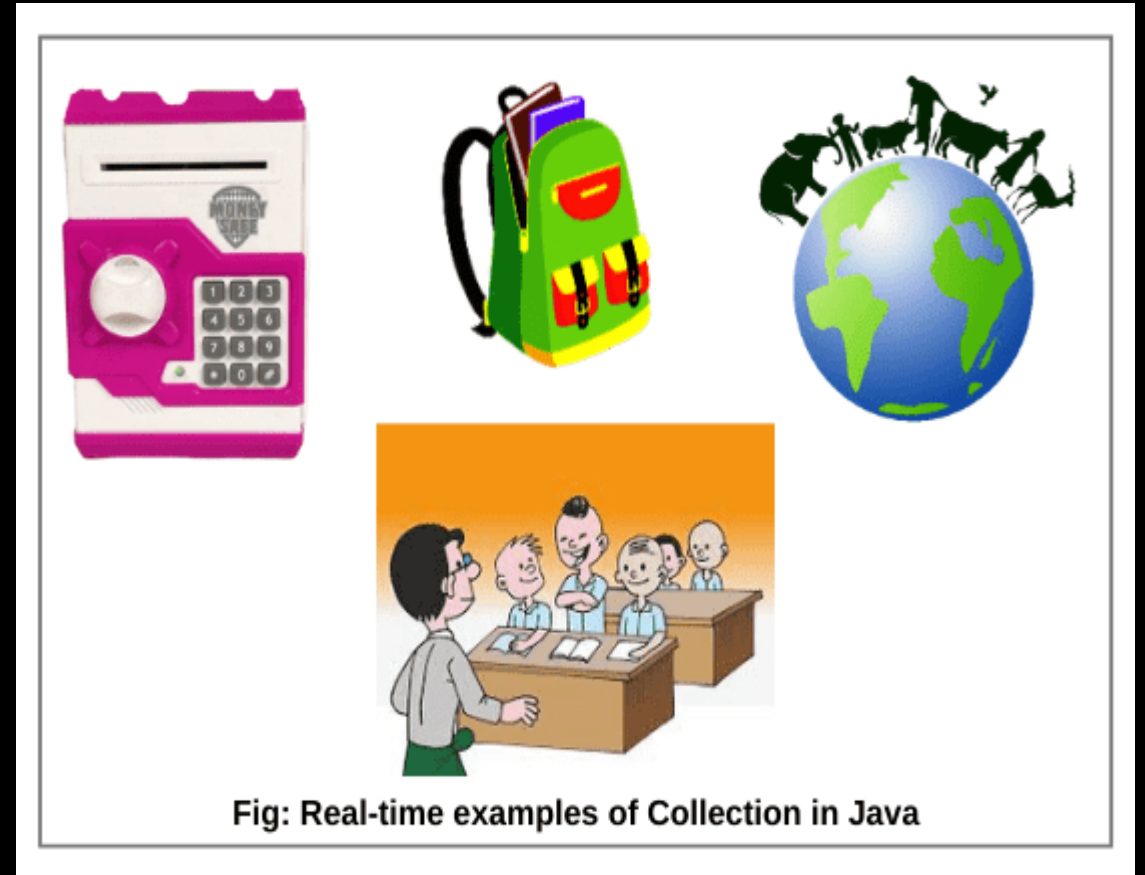
- **Reduces programming effort by providing useful data structures and algorithms.**
- **Increases program speed and quality since the implementation of each interface is interchangeable.**
- **Allows interoperability among different APIs.**
- **Extending or adapting a collection is easy.**

The Collection Classes

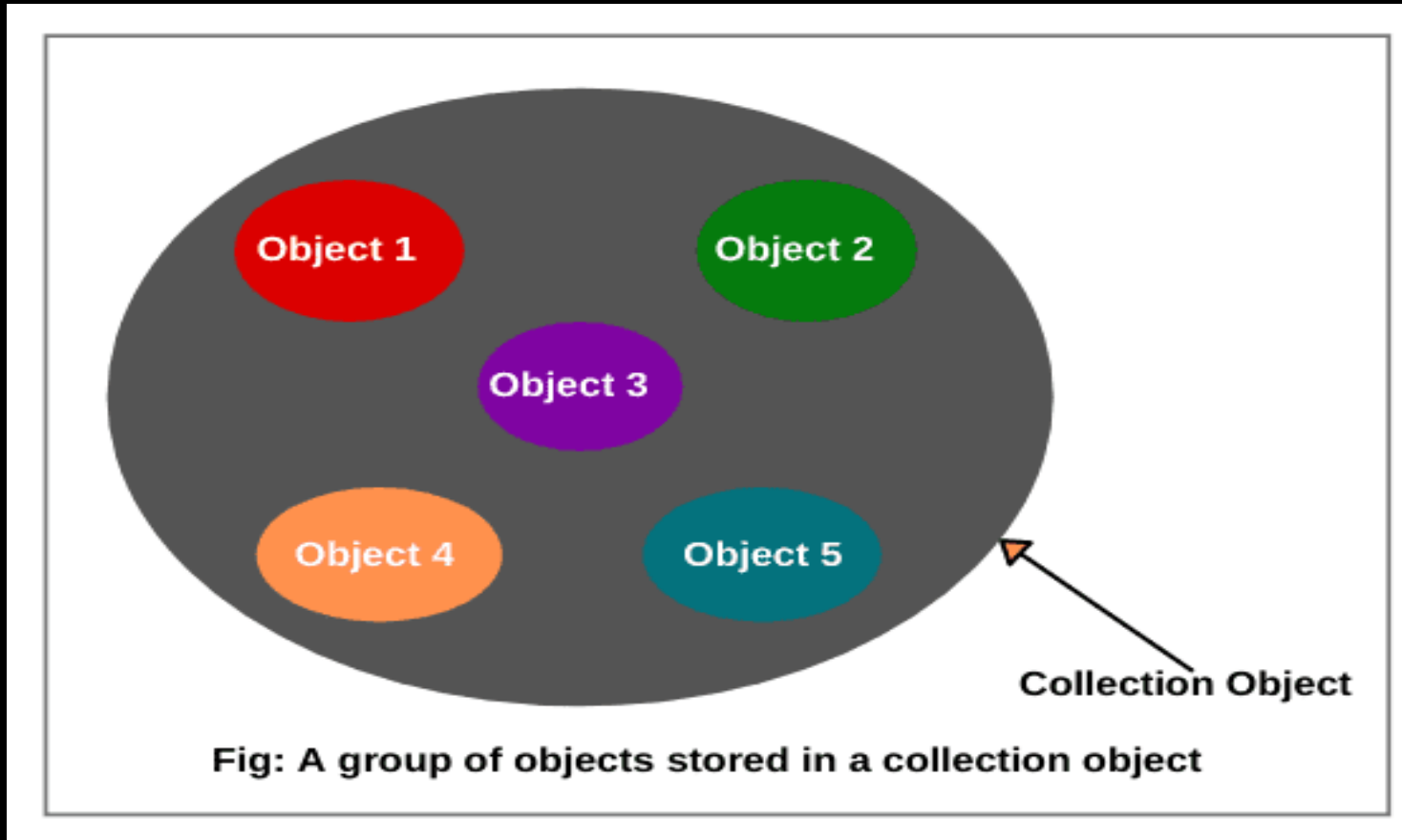
- Java provides a set of standard collection classes that implement Collection interfaces.
- The Collection interface (java.util.Collection) and Map interface (java.util.Map) are two main root interfaces of Java collection classes.
- The **java.util** package contains all the classes and interfaces for Collection framework.

Real-time Examples - Collections

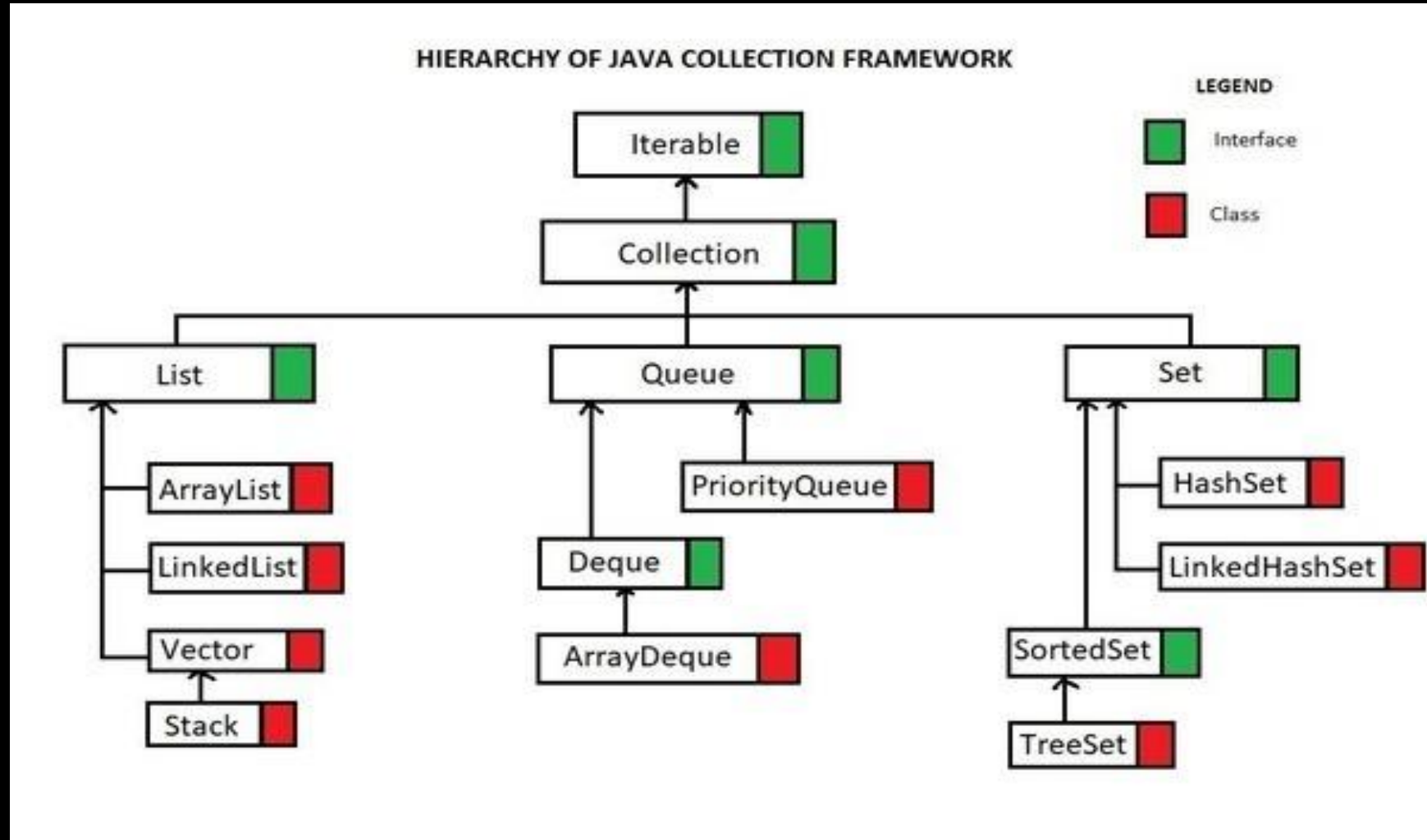
- In childhood, you had a kiddy bank. In the kiddy bank, you had collected a lot of coins. This kiddy bank is called collection and the coins are nothing but objects.
- During school time, you put all the important books in the school bag before going to school. Here Schoolbag is a collection and books are objects.
- A classroom is a collection of students. So, the classroom is nothing but a collection and students are objects.



Collections – Group of Objects



Hierarchy of Collection Framework



Methods of Collection interface

Method	Description
public boolean add(Object element)	is used to insert an element in this collection.
public boolean addAll(Collection c)	is used to insert the specified collection elements in the invoking collection.
public boolean remove(Object element)	is used to delete an element from this collection.
public boolean removeAll(Collection c)	is used to delete all the elements of specified collection from the invoking collection.
public boolean retainAll(Collection c)	is used to delete all the elements of invoking collection except the specified collection.
public int size()	return the total number of elements in the collection.
public void clear()	removes the total no of element from the collection.
public boolean contains(Object element)	is used to search an element.
public boolean containsAll(Collection c)	is used to search the specified collection in this collection.
public Iterator iterator()	returns an iterator.
public Object[] toArray()	converts collection into array.
public boolean isEmpty()	checks if collection is empty.
public boolean equals(Object element)	matches two collection.
public int hashCode()	returns the hashcode number for collection.

Iterator interface

- Iterator interface provides the facility of iterating the elements in forward direction only.
- Iterator is an interface available in Collection framework in java.util package.
- It is a Java Cursor used to iterate a collection of objects.
- It is used to traverse a collection object elements one by one.
- It is available since Java 1.2 Collection Framework.
- It is applicable for all Collection classes. So it is also known as Universal Java Cursor.
- It supports both READ and REMOVE Operations.
- Compare to Enumeration interface, Iterator method names are simple and easy to use.

Iterator Methods

Method	Description
public boolean hasNext()	It returns true if iterator has more elements.
public Object next()	It returns the element and moves the cursor pointer to the next element.
public void remove()	It removes the last elements returned by the iterator. It is rarely used.



Iterator Demo

- Get an Iterator object from a Collection.
- Each and every Collection class has the following iterator() method to iterate it's elements.

Iterator<E> iterator()

Iterator<String> namesIterator = names.iterator();

Ways to iterate the elements of collection

- **The Classic For Loop**
- **The Iterator Method**
- **The Enhanced For Loop**
- **The forEach Method with Lambda Expressions**

```
for (int i = 0; i < listNames.size(); i++) {  
    String aName = listNames.get(i);  
    System.out.println(aName); }
```

```
Iterator<String> iterator = listNames.iterator();  
while (iterator.hasNext()) {  
    String aName = iterator.next();  
    System.out.println(aName);}
```

```
for (String aName : listNames) {  
    System.out.println(aName);  
}
```

```
listNames.forEach(name ->  
    System.out.println(name));
```

When to Use ForEachLoop ?

- When you only need to iterate over elements without modifying the collection.
- When working with collections or arrays where element removal is not required.
- When code simplicity and readability are prioritized.

When to Use Iterator ?

- When you require more control over the iteration process, such as removing elements during iteration.
- When working with collections that do not support `ForEachLoop`, or custom classes implementing the `Iterable` interface.
- When fine-grained manipulation or complex iteration logic is necessary.

When to use for loop in collection java ?

- When iterating over an array or a collection with a known size.
- When performing a fixed number of iterations.
- When you need to access the index of each element.
- In contrast, an iterator or enhanced for loop is more suitable when iterating over a collection without knowing its size or when you only need to access the elements, not their indices.

List Interface

- List interface is the child interface of Collection interface.
- It inhibits a list type data structure in which we can store the ordered collection of objects. It can have duplicate values.
- List interface is implemented by the classes ArrayList, LinkedList, Vector, and Stack.
- To instantiate the List interface, we must use :
 - `List <data-type> list1= new ArrayList();`
 - `List <data-type> list2 = new LinkedList();`
 - `List <data-type> list3 = new Vector();`
 - `List <data-type> list4 = new Stack();`

ArrayList 2-1

- An ArrayList object is a variable length array of object references.
- It is used to create dynamic arrays.
- Extends AbstractList and implements List interface.
- ArrayLists are created with an initial size.
- As elements are added, size increases and the array expands.
- It gives better performance when accessing and iterating through objects.

125



```
PlayersList()
```

```
player
```

```
void display() {
```

```
    System.out.println("**
```

```
    + "*****");
```

```
    System.out.println("Ret
```

```
    System.out.println("**
```

```
    + "*****");
```

```
    System.out.println();
```

```
    for (int ctr = 0; ctr <
```

```
        System.out.print("
```

```
    }
```

```
} void extract() {
```

```
    System.out.println("*****
```

```
    + "*****");
```

```
    System.out.println("Extract a sublist and "
```

```
    + "then print the new List ");
```

```
    System.out.println("*****
```

```
    + "*****");
```

```
    System.out.println();
```

```
    subListObj = playerArray.subList(5, playerArray.size());
```

```
    System.out.println("New Sub-List from index 5 to "
```

```
    + playerArray.size() + " is : " + subListObj);
```

```
    System.out.println();
```

```
}
```

```
void search() {
```

```
    System.out.println();
```

```
    System.out.println("*****
```

```
    + "*****");
```

```
    System.out.println("Search for an object and return "
```

```
    + "the first and last position");
```

```
    System.out.println("*****
```

```
    + "*****");
```

```
    System.out.println();
```

```
    System.out.println("First occurrence of the String"
```

```
    + " \"Serena\" is at position " + playerArray.indexOf("Serena"));
```

```
    System.out.println("Last occurrence of the String"
```

```
    + " \"Serena\" is at position " + playerArray.lastIndexOf("Serena"));
```

```
    System.out.println();
```

hold value of a particular
datatype only

new list with given size.
new automatically as
ded to the array list.

Extracting a portion of the
ArrayList and storing it
in the second
ArrayList by using the
subList() method

Demonstra

Searching the ArrayList for the
first and last occurrence of an
object using the indexOf()
and lastIndexOf() method

Generics in Collections

- The **Java Generics** programming is introduced in J2SE 5 to deal with type-safe objects.
- It makes the code stable by detecting the bugs at compile time.
- Before generics, we can store any type of objects in the collection, i.e., non-generic.
- Now generics force the java programmer to store a specific type of objects.

Advantage of Java Generics in Collections

- **Type-safety:** We can hold only a single type of objects in generics. It doesn't allow to store other objects.
- **Without Generics,** we can store any type of objects.

```
List list = new ArrayList();
```

```
list.add(10);
```

```
list.add("10");
```

//With Generics, it is required to specify the type of object we need to store.

```
List<Integer> list = new ArrayList<Integer>();
```

```
list.add(10);
```

```
list.add("10");// compile-time error
```

Advantage of Java Generics in Collections

- **Type casting is not required: There is no need to typecast the object.**
- **Before Generics, we need to type cast.**

```
List list = new ArrayList();
```

```
list.add("hello");
```

```
String s = (String) list.get(0); //typecasting
```

After Generics, we don't need to typecast the object.

```
List<String> list = new ArrayList<String>();
```

```
list.add("hello");
```

```
String s = list.get(0);
```

Advantage of Java Generics in Collections

- **Compile-Time Checking:** It is checked at compile time so problem will not occur at runtime. The good programming strategy says it is far better to handle the problem at compile time than runtime.

```
List<String> list = new ArrayList<String>();
```

```
list.add("hello");
```

```
list.add(32); // Compile Time Error
```

Syntax to use generic collection

ClassOrInterface<Type>

Example to use Generics in java

ArrayList<String>

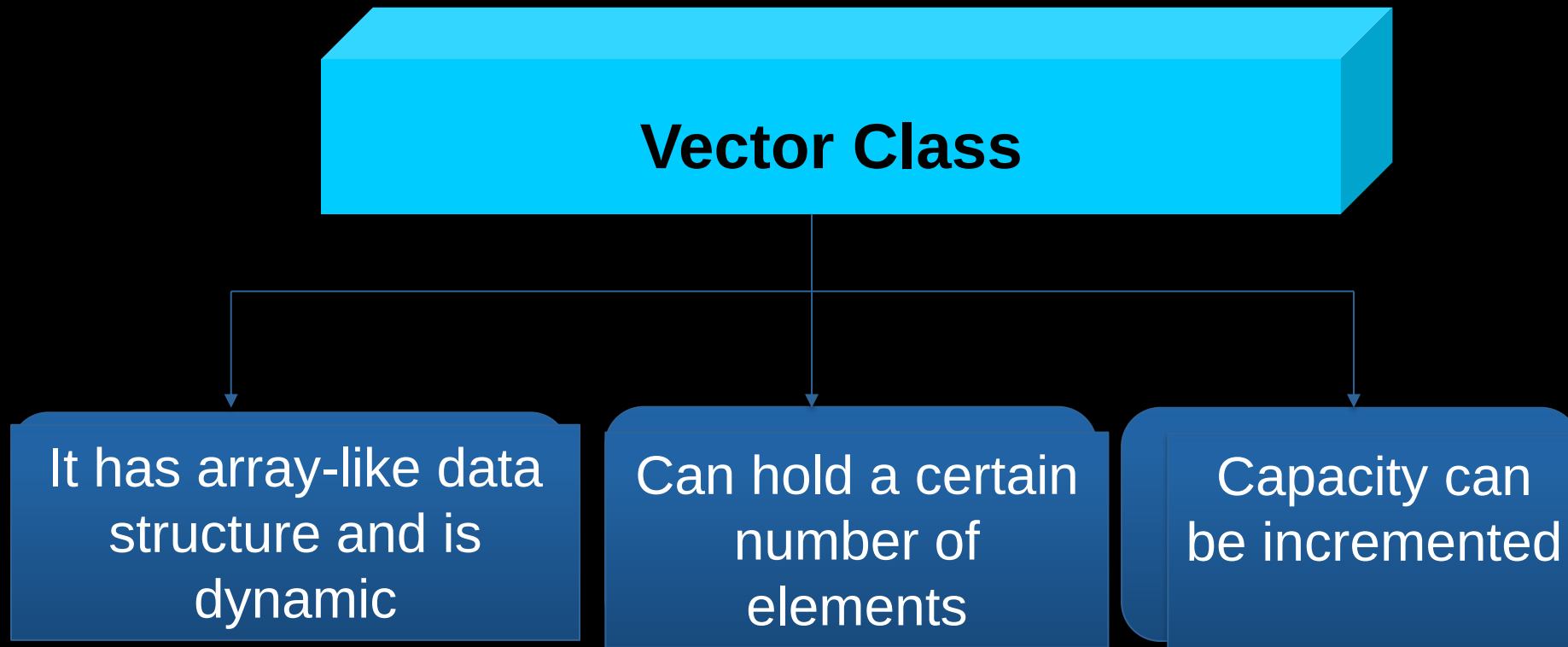
Collections Class

- **Collections is an utility class in java.util package.**
- **It consists of only static methods which are used to operate on objects of type Collection.**
- **For example, it has the method to find the**
 - maximum element in a collection
 - sort the collection
 - search for a particular element in a collection.

Collections –Utility Class -Methods

Collections.max()	This method returns maximum element in the specified collection.
Collections.min()	This method returns minimum element in the given collection.
Collections.sort()	This method sorts the specified collection.
Collections.shuffle()	This method randomly shuffles the elements in the specified collection.
Collections.synchronizedCollection()	This method returns synchronized collection backed by the specified collection.
Collections.binarySearch()	This method searches the specified collection for the specified object using binary search algorithm.
Collections.disjoint()	This method returns true if two specified collections have no elements in common.
Collections.copy()	This method copies all elements from one collection to another collection.
Collections.reverse()	This method reverses the order of elements in the specified collection.

Vector class 3-1



Vector class 3-2

Constructor	Description
<code>Vector()</code>	Creates an empty vector. Its size is 10 and capacity increment is 0.
<code>Vector(int initialCap)</code>	Creates an empty Vector with initial capacity specified by <i>initialCap</i> and capacity increment is 0.
<code>Vector (int initialCap, int inc)</code>	Creates an empty Vector with initial capacity specified by <i>initialCap</i> and capacity increment specified by <i>inc</i> .
<code>Vector (Collection c)</code>	Creates a new Vector consisting of elements of the given collection, in the order in which, they are returned by the collections iterator.

Difference between ArrayList and Vector

ArrayList	Vector
ArrayList is not synchronized .	Vector is synchronized
ArrayList increments 50% of current array size if number of element exceeds from its capacity.	Vector increments 100% means doubles the array size if total number of element exceeds than its capacity.
ArrayList is not a legacy class, it is introduced in JDK 1.2.	Vector is a legacy class.
ArrayList is fast because it is non-synchronized.	Vector is slow because it is synchronized i.e. in multithreading environment, it will hold the other threads in runnable or non-runnable state until current thread releases the lock of object.
ArrayList uses Iterator interface to traverse the elements.	Vector uses Enumeration interface to traverse the elements. But it can use Iterator also.

```

void otherDetails() {
    System.out.println("\nFirst Element = "
        + preciousVector.firstElement());
    System.out.println("Default Capacity = "
        + preciousVector.capacity());
    System.out.println("Last Element = "
        + preciousVector.lastElement());
}

```

cannot be added to a
Creating an object of
the Vector class

```

void add() {
    preciousVector.addElement("Jade");
    preciousVector.addElement("Topaz");
    preciousVector.addElement("Turquoise");
    preciousVector.addElement("Emerald");
}

```

```

System.out.println("*****");
System.out.println("Searching Contents :");
System.out.println("*****");
String searchStone = "Diamond";
if (preciousVector.contains(searchStone)) {
    System.out.println("Found " + searchStone
        + " at index " + preciousVector.indexOf(searchStone));
}

```

Inserting elements to the
object of Vector class by
using the
insertElementAt()
method

```

void insert() {
    preciousVector.insertElementAt("Diamond", 0);
    preciousVector.insertElementAt("Opal", 4);
}

```

Usage of other methods like
firstElement(),
capacity(),
lastElement()

Demonstration: Example 8

Stack Class

- Java provides an inbuilt object type called **Stack**.
- It is a collection that is based on the last in first out (LIFO) principle.
- Stack only defines the default constructor, which creates an empty stack.
- Stack is a subclass of Vector that implements a standard last-in, first-out stack.

Realtime Example of Stack in Java



A stack of books



A pile of plates



A stack of dvd/cd

Fig: Realtime examples of Stack



Original stack

45

Pop element



After pop()



After push(50)

Fig: Push and Pop operations in Java Stack

Stack Methods

It extends Vector class with five methods that allow a vector to be treated as a stack.

- Object push(Object element) : **Pushes an element on the top of the stack.**
- Object pop() : **Removes and returns the top element of the stack. An 'EmptyStackException' exception is thrown if we call pop() when the invoking stack is empty.**
- Object peek() : **Returns the element on the top of the stack, but does not remove it.**
- boolean empty() : **It returns true if nothing is on the top of the stack. Else, returns false.**
- int search(Object element) : **It determines whether an object exists in the stack. If the element is found, it returns the position of the element from the top of the stack. Else, it returns -1.**

Features of Stack class

- **Stack is a group of elements with “last-in, first-out” retrieval.**
- **In the Java stack, all operations take place at the top of the stack.**
- **Push operation inserts an element to the top of stack.**
- **Pop operation removes an element from the stack and returns it.**
- **Stack class is synchronized. That means it is thread-safe.**
- **Null elements are allowed into the stack.**
- **Duplicate elements are allowed into the stack.**

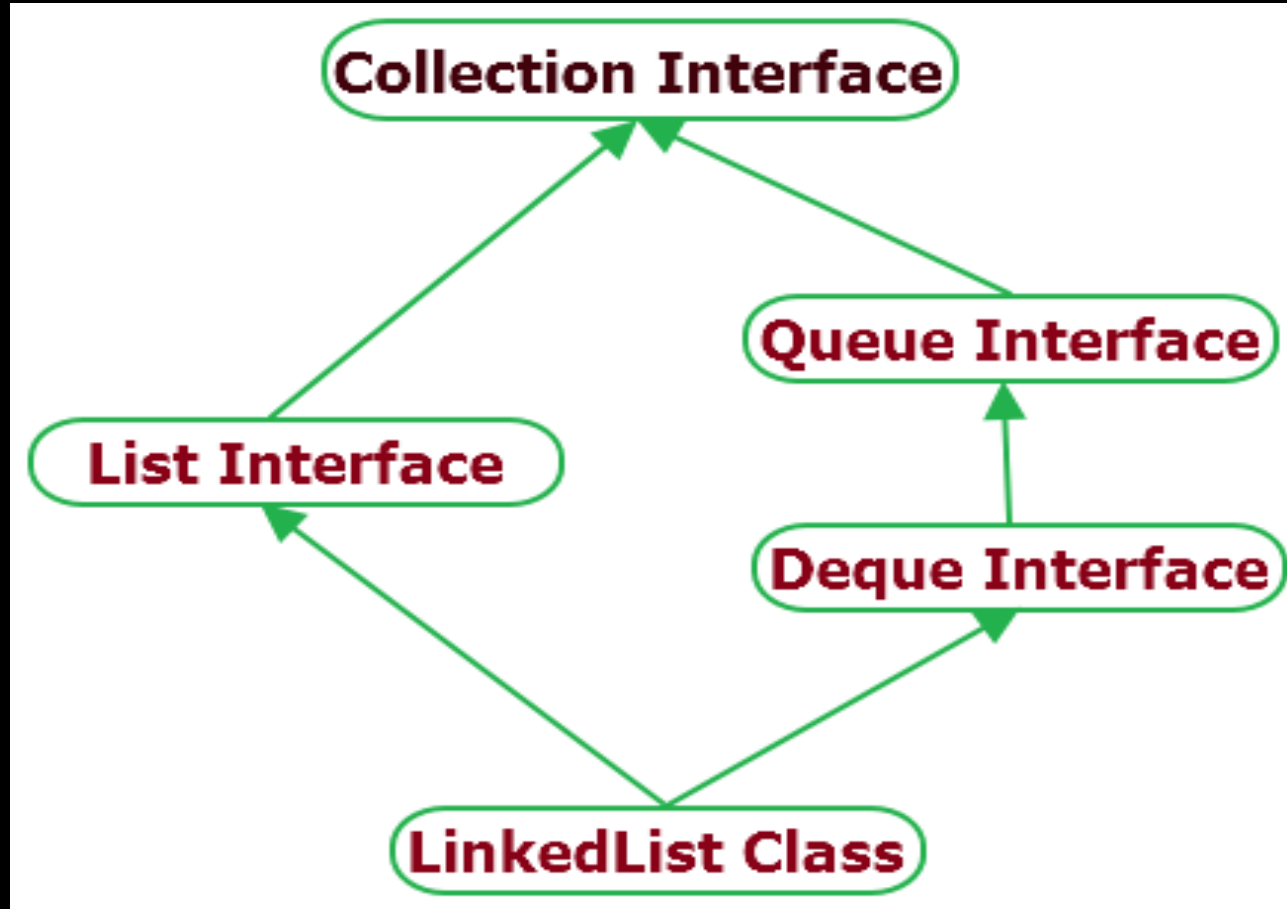
Linked List

- **LinkedList** is a generic class that extends the **AbstractSequentialList** and implements **List**, **Queue**, and **Deque** interfaces.
- It basically is an implementation of a type of linked list data structure that facilitates the storage of elements.
- The contents of this storage can grow and shrink at run time as per the requirement.
- Quite visibly, it is not very different from the other List classes, such as **ArrayList**.
- But, the point of difference is in how the list is maintained at the core.

Linked List

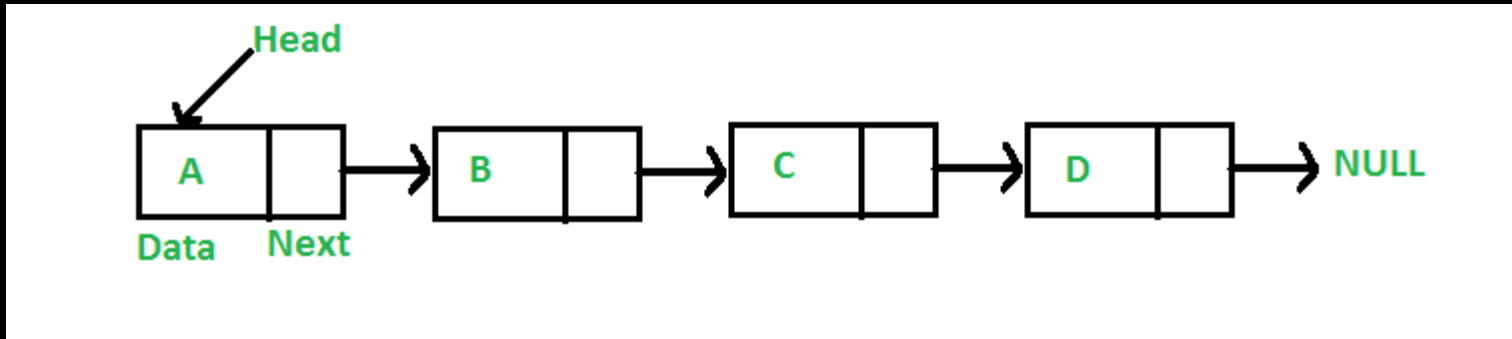
- **LinkedList class uses doubly linked list to store the elements.**
- **It provides a linked-list data structure.**
- **LinkedList class can contain duplicate elements.**
- **LinkedList class maintains insertion order.**
- **LinkedList class is non synchronized.**
- **LinkedList class, manipulation is fast because no shifting needs to be occurred.**
- **LinkedList class can be used as list, stack or queue.**

Linked List Hierarchy



Linked Lists

- Like arrays, Linked List is a linear data structure.
- Unlike arrays, linked list elements are not stored at contiguous location; the elements are linked using pointers



Why Linked List

Arrays can be used to store linear data of similar types, but arrays have following limitations.

- The size of the arrays is fixed: So we must know the upper limit on the number of elements in advance. Also, generally, the allocated memory is equal to the upper limit irrespective of the usage.
- Inserting a new element in an array of elements is expensive, because room has to be created for the new elements and to create room existing elements have to be shifted.

Pros & Cons

Pros:

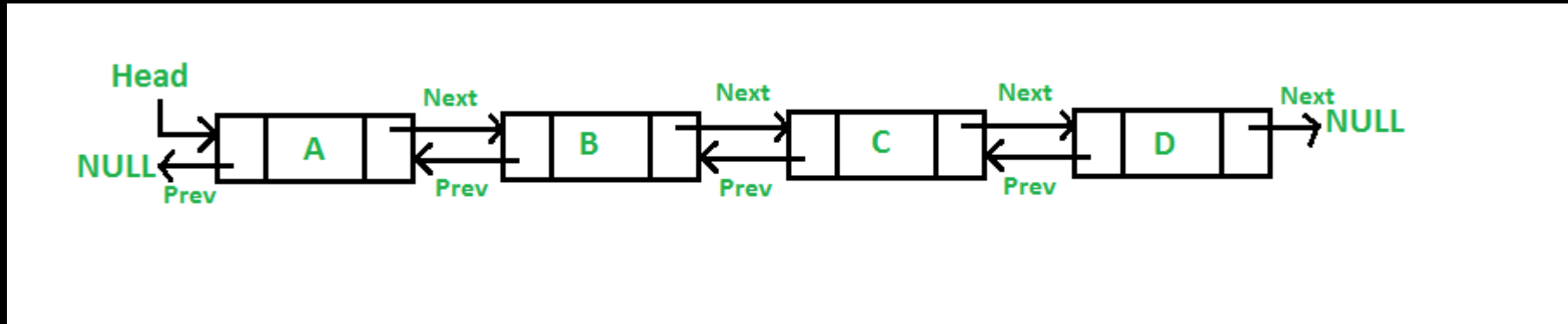
- 1) Dynamic size
- 2) Ease of insertion/deletion

Cons:

- 1) Random access is not allowed. We have to access elements sequentially starting from the first node. So we cannot do binary search with linked lists.
- 2) Extra memory space for a pointer is required with each element of the list.

Doubly Linked List

- A **Doubly Linked List (DLL)** contains an extra pointer, typically called *previous pointer*, together with next pointer and data which are there in singly linked list.



Advantages over singly linked list

- A DLL can be traversed in both forward and backward direction.
- The delete operation in DLL is more efficient if pointer to the node to be deleted is given.
- In singly linked list, to delete a node, pointer to the previous node is needed. To get this previous node, sometimes the list is traversed. In DLL, we can get the previous node using previous pointer.

Insertion

A node can be added in four ways

- **At the front of the DLL**
- **After a given node.**
- **At the end of the DLL**
- **Before a given node.**

LinkedList class declaration

```
public class LinkedList<E> extends AbstractSequentialList<E>  
implements List<E>, Deque<E>, Cloneable, Serializable
```

Constructor	Description
LinkedList()	It is used to construct an empty list.
LinkedList(Collection c)	It is used to construct a list containing the elements of the specified collection, in the order they are returned by the collection's iterator.

Methods of LinkedList

Method	Description
void add(int index, Object element)	It is used to insert the specified element at the specified position index in a list.
void addFirst(Object o)	It is used to insert the given element at the beginning of a list.
void addLast(Object o)	It is used to append the given element to the end of a list.
int size()	It is used to return the number of elements in a list
boolean add(Object o)	It is used to append the specified element to the end of a list.
boolean contains(Object o)	It is used to return true if the list contains a specified element.
boolean remove(Object o)	It is used to remove the first occurrence of the specified element in a list.
Object getFirst()	It is used to return the first element in a list.
Object getLast()	It is used to return the last element in a list.
int indexOf(Object o)	It is used to return the index in a list of the first occurrence of the specified element, or -1 if the list does not contain any element.
int lastIndexOf(Object o)	It is used to return the index in a list of the last occurrence of the specified element, or -1 if the list does not contain any element.

Linked List - Demo

```
import java.util.*;

public class TestCollection{
    public static void main(String args[]){

        LinkedList<String> al=new LinkedList<String>();

        al.add("Mike");
        al.add("John");
        al.add("Sandra");
        al.add("Jovel");
```

```
        Iterator<String> itr=al.iterator();

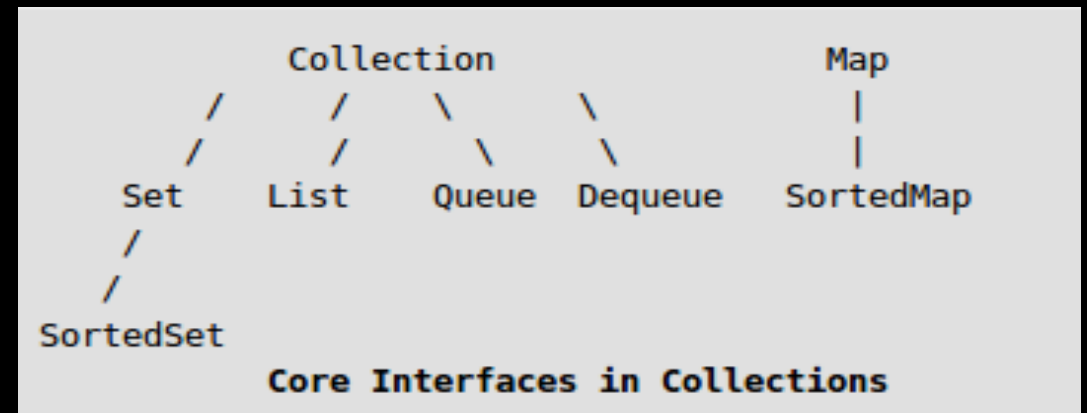
        while(itr.hasNext()){
            System.out.println(itr.next());
        } } }
```

ArrayList vs LinkedList

- **Insert and Remove operations give good performance in LinkedList compared to ArrayList.**
- **If there is a requirement of frequent addition and deletion in application then LinkedList is a best choice.**
- **Search (get method) operations are fast in Arraylist but not in LinkedList .**
- **So If there are less add and remove operations and more search operations requirement, ArrayList would be your best bet.**

List Interface

- Java.util.List is a child interface of [Collection](#).
- List is an ordered collection of objects in which duplicate values can be stored.
- Since List preserves the insertion order it allows positional access and insertion of elements.
- List Interface is implemented by ArrayList, LinkedList, Vector and Stack classes.



List Interface

- Unlike sets, lists typically allow duplicate elements.
- It contains methods to insert and delete elements in index basis.
- Since List preserves the insertion order it allows positional access and insertion of elements.
- Creating List Objects .

```
List a = new ArrayList();  
List b = new LinkedList();  
List c = new Vector();  
List d = new Stack();
```

List – ordered Collection

In addition to the operations inherited from Collection, the List interface includes operations for the following:

- **Positional access** — manipulates elements based on their numerical position in the list. This includes methods such as `get`, `set`, `add`, `addAll`, and `remove`.
- **Search** — searches for a specified object in the list and returns its numerical position. Search methods include `indexOf` and `lastIndexOf`.
- **Iteration** — extends `Iterator` semantics to take advantage of the list's sequential nature. The `listIterator` methods provide this behavior.
- **Range-view** — The `sublist` method performs arbitrary range operations on the list.

List Interface declaration

- **public interface List<E> extends Collection<E>**

Methods	Description
void add(int index,Object element)	It is used to insert element into the invoking list at the index passed in the index.
boolean addAll(int index,Collection c)	It is used to insert all elements of c into the invoking list at the index passed in the index.
object get(int index)	It is used to return the object stored at the specified index within the invoking collection.
object set(int index,Object element)	It is used to assign element to the location specified by index within the invoking list.
object remove(int index)	It is used to remove the element at position index from the invoking list and return the deleted element.
ListIterator listIterator()	It is used to return an iterator to the start of the invoking list.
ListIterator listIterator(int index)	It is used to return an iterator to the invoking list that begins at the specified index.

List Example

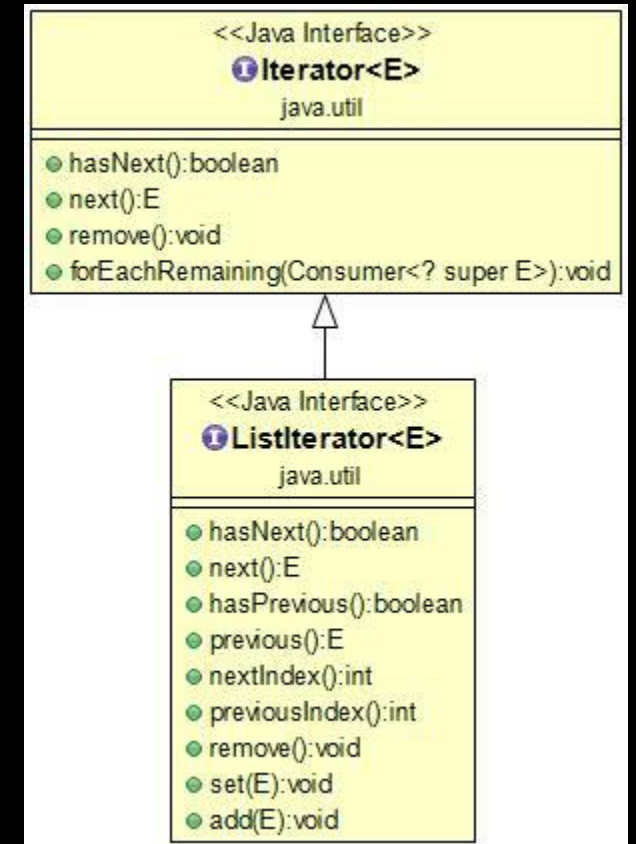
```
public class ListExample{  
    public static void main(String args[]){  
        ArrayList<String> al=new ArrayList<String>();  
        al.add("Jean");  
        al.add("Jenny");  
        al.add("Jim");  
        al.add(1,"Jack");  
  
        System.out.println("Element at 2nd position: "+al.get(2));  
        for(String s:al) {  
            System.out.println(s); } } }
```


Java ListIterator Interface

- An iterator for lists that allows the programmer to traverse the list.
- It performs iteration in either direction, modify the list during iteration, and obtain the iterator's current position in the list.
- It extends Iterator interface.
- It is useful only for List implemented classes.

ListIterator

- Unlike Iterator, It supports all four operations:
- CRUD (CREATE, READ, UPDATE and DELETE).
- Unlike Iterator, It supports both Forward Direction and Backward Direction iterations.
- It is a Bi-directional Iterator.
- It has no current element; its cursor position always lies between the element that would be returned by a call to previous() and the element that would be returned by a call to next().



ListIterator Interface declaration

- **public interface ListIterator<E> extends Iterator<E>**

Methods	Description
boolean hasNext()	This method return true if the list iterator has more elements when traversing the list in the forward direction.
Object next()	This method return the next element in the list and advances the cursor position.
boolean hasPrevious()	This method return true if this list iterator has more elements when traversing the list in the reverse direction.
Object previous()	This method return the previous element in the list and moves the cursor position backwards.

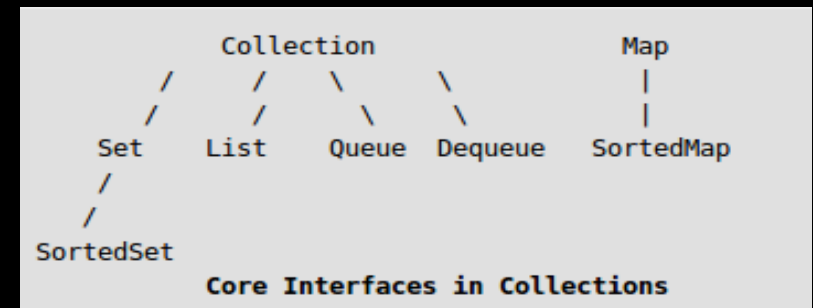
Example of ListIterator Interface

```
public class ListIteratorDemo{  
    public static void main(String args[]){  
        ArrayList<String> al=new  
        ArrayList<String>();  
        al.add("Jim");  
        al.add("Jerry");  
        al.add("John");  
        al.add(1,"Jive");  
        System.out.println("element at 2nd  
position: "+al.get(2));
```

```
ListIterator<String> itr=al.listIterator();  
System.out.println("traversing elements in  
forward direction...");  
while(itr.hasNext()){  
    System.out.println(itr.next());  
}  
System.out.println("traversing elements in  
backward direction...");  
while(itr.hasPrevious()){  
    System.out.println(itr.previous()); } } }
```

The Set Interface

- A Set is a Collection that cannot contain duplicate elements.
- The Set interface contains only methods inherited from Collection and adds the restriction that duplicate elements are prohibited.
- Set also adds a stronger contract on the behavior of the equals and hashCode operations, allowing Set instances to be compared meaningfully even if their implementation types differ.



List vs Set

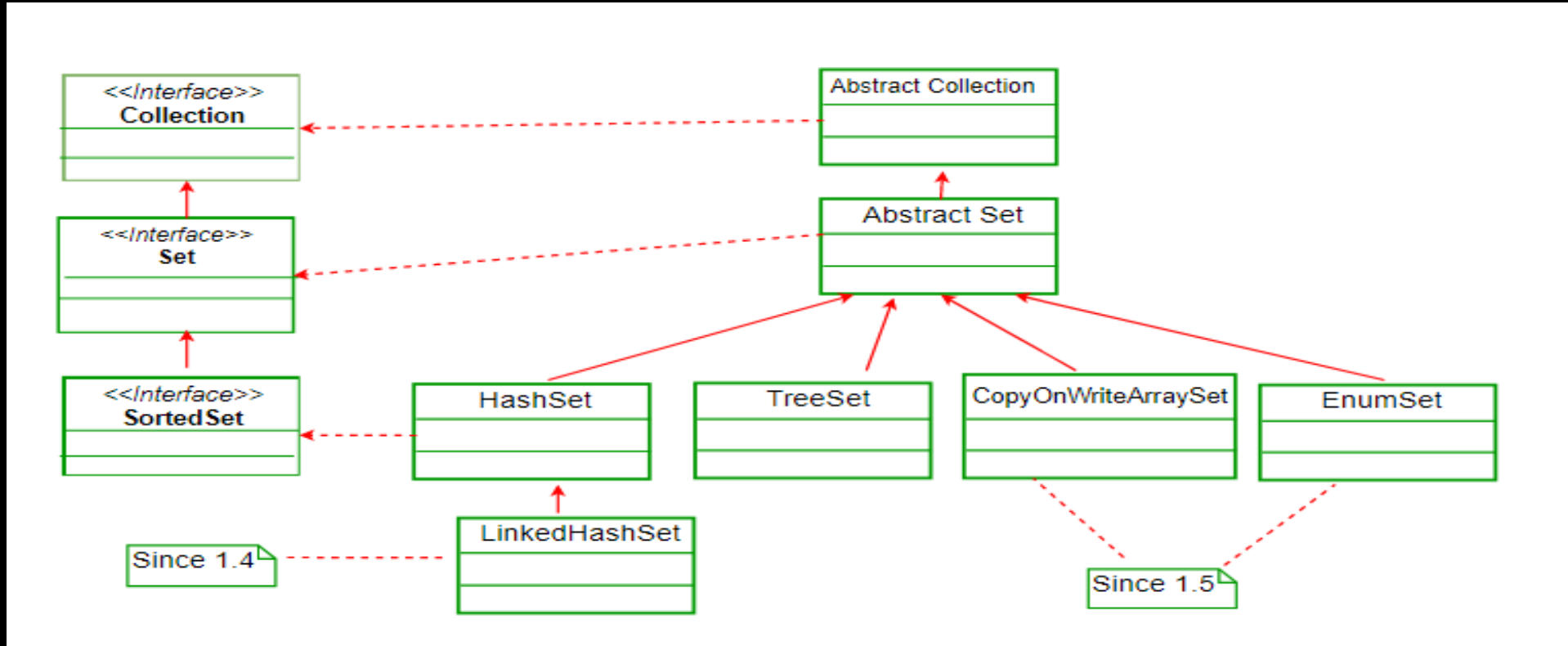
List	Set
Is an Ordered grouping of elements.	Is an Unordered grouping of elements.
List is used to collection of elements with duplicates.	Set is used to collection of elements without duplicates.
New methods are defined inside List interface.	No new methods are defined inside Set interface, so we have to use Collection interface methods only with Set subclasses.
Use list for storing non-unique objects as per insertion order.	Use set for storing unique objects in random order

Hash Set Class

- **Java HashSet class is used to create a collection that uses a hash table for storage.**
- **It inherits the AbstractSet class and implements Set interface.**
- **Objects that you insert in HashSet are not guaranteed to be inserted in same order. Objects are inserted based on their hash code.**
- **HashSet stores the elements by using a mechanism called hashing.**
- **HashSet contains unique elements only.**

HashSet Hierarchy

- The HashSet class extends AbstractSet class which implements Set interface.
- The Set interface inherits Collection and Iterable interfaces in hierarchical order.



HashSet class declaration

- **public class HashSet<E> extends AbstractSet<E>
implements Set<E>, Cloneable, Serializable**

Constructor	Description
HashSet()	It is used to construct a default HashSet.
HashSet(Collection c)	It is used to initialize the hash set by using the elements of the collection c.
HashSet(int capacity)	It is used to initialize the capacity of the hash set to the given integer value capacity. The capacity grows automatically as elements are added to the HashSet.

Methods of Java HashSet class

Methods	Description
void clear()	It is used to remove all of the elements from this set.
boolean contains(Object o)	It is used to return true if this set contains the specified element.
boolean add(Object o)	It is used to adds the specified element to this set if it is not already present.
boolean isEmpty()	It is used to return true if this set contains no elements.
boolean remove(Object o)	It is used to remove the specified element from this set if it is present.
Object clone()	It is used to return a shallow copy of this HashSet instance: the elements themselves are not cloned.
Iterator iterator()	It is used to return an iterator over the elements in this set.
int size()	It is used to return the number of elements in this set.

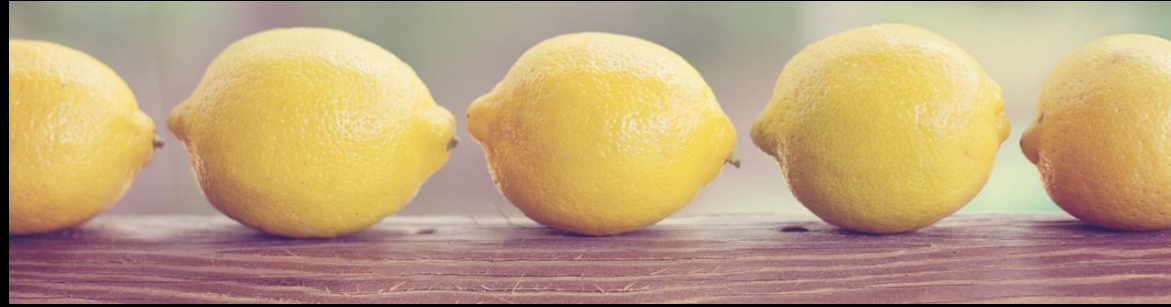
HashSet Example

```
public class HashSetExample {  
    public static void main(String args[]) {  
        HashSet<String> hset = new HashSet<String>();  
  
        hset.add("Apple");  
        hset.add("Mango");  
        hset.add("Grapes");  
        hset.add("Orange");  
        hset.add("Fig");  
    }  
}
```

```
//Addition of duplicate elements  
hset.add("Apple");  
hset.add("Mango");  
//Addition of null values  
hset.add(null);  
hset.add(null);  
  
//Displaying HashSet elements  
System.out.println(hset);  
}  
}
```

[null, Mango, Grapes, Apple, Orange, Fig]

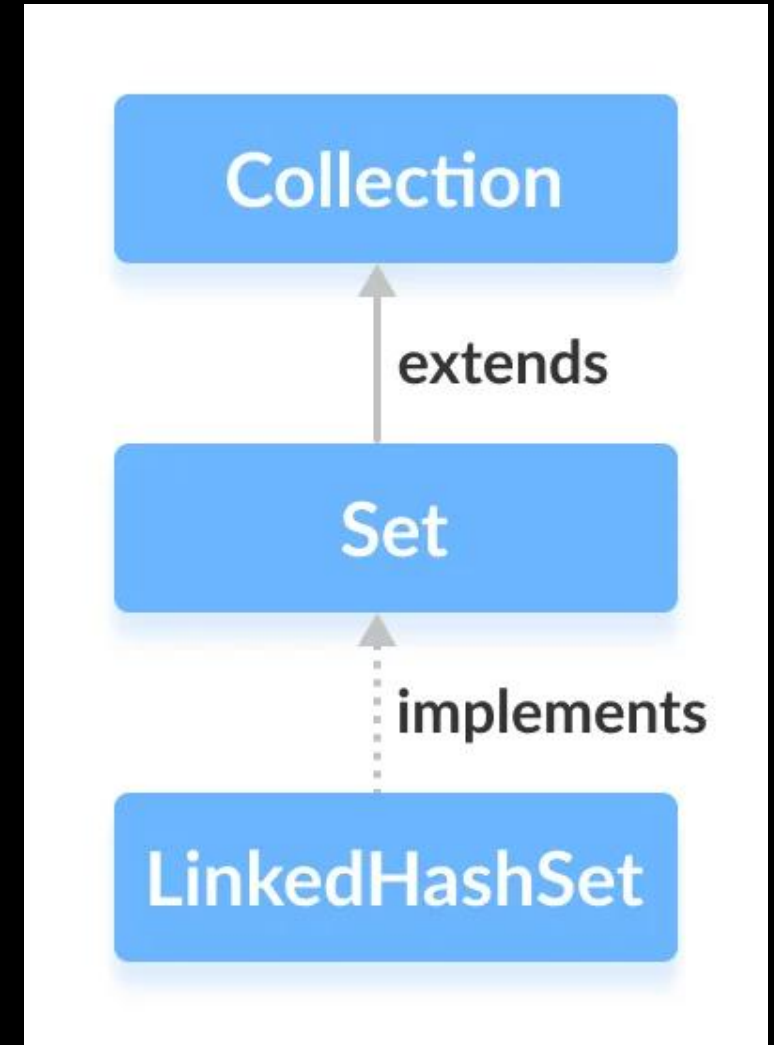
LinkedHashSet



- The LinkedHashSet is an ordered version of HashSet that maintains a doubly-linked List across all elements.
- When the iteration order is needed to be maintained this class is used.
- When iterating through a HashSet the order is unpredictable, while a LinkedHashSet lets us iterate through the elements in the order in which they were inserted.
- When cycling through LinkedHashSet using an iterator, the elements will be returned in the order in which they were inserted.

LinkedHashSet

- The LinkedHashSet class of the Java collections framework provides functionalities of both the hashtable and the linked list data structure.
- Elements of LinkedHashSet are stored in hash tables similar to HashSet.
- However, linked hash sets maintain a doubly-linked list internally for all of its elements.
- It provides all optional set operations and permits null elements.
- Contains unique elements only like HashSet.
- It extends the HashSet class and implements the Set interface.
- It maintains insertion order.



Declaration & Constructors of LinkedHashMap Class

```
public class LinkedHashMap<E> extends HashSet<E> implements Set<E>, Cloneable, Serializable
```

Constructor	Description
HashSet()	It is used to construct a default HashSet.
HashSet(Collection c)	It is used to initialize the hash set by using the elements of the collection c.
LinkedHashSet(int capacity)	It is used to initialize the capacity of the linked hash set to the given integer value capacity.
LinkedHashSet(int capacity, float fillRatio)	It is used to initialize both the capacity and the fill ratio (also called load capacity) of the hash set from its argument.

- By default, the capacity of the linked hash set will be 16
- the load factor will be 0.75 (75%)

LinkedHashSet Methods

Method	Purpose
<code>public boolean add(Object o)</code>	Adds an object to a LinkedHashSet if already not present in HashSet.
<code>public boolean remove(Object o)</code>	Removes an object from LinkedHashSet if found in HashSet.
<code>public boolean contains(Object o)</code>	Returns true if object found else return false
<code>public boolean isEmpty()</code>	Returns true if LinkedHashSet is empty else return false
<code>public int size()</code>	Returns number of elements in the LinkedHashSet
<code>splititerator()</code>	Creates a late-binding and fail-fast Spliterator over the elements in this set.

LinkedHashSet Demo - Union of Sets

```
import java.util.LinkedHashSet;
class Main {
public static void main(String[] args) {
LinkedHashSet<Integer> evenNumbers = new LinkedHashSet<>();
evenNumbers.add(2);
evenNumbers.add(4);
System.out.println("LinkedHashSet1: " + evenNumbers);

LinkedHashSet<Integer> numbers = new LinkedHashSet<>();
numbers.add(1);
numbers.add(3);
System.out.println("LinkedHashSet2: " + numbers);

// Union of two set numbers.addAll(evenNumbers);
System.out.println("Union is: " + numbers); } }
```

Output

```
LinkedHashSet1: [2, 4]
LinkedHashSet2: [1, 3]
Union is: [1, 3, 2, 4]
```


Other Methods Of LinkedHashSet

Method	Description
clone()	Creates a copy of the LinkedHashSet
contains()	Searches the LinkedHashSet for the specified element and returns a boolean result
isEmpty()	Checks if the LinkedHashSet is empty
size()	Returns the size of the LinkedHashSet
clear()	Removes all the elements from the LinkedHashSet

LinkedHashSet Vs. HashSet

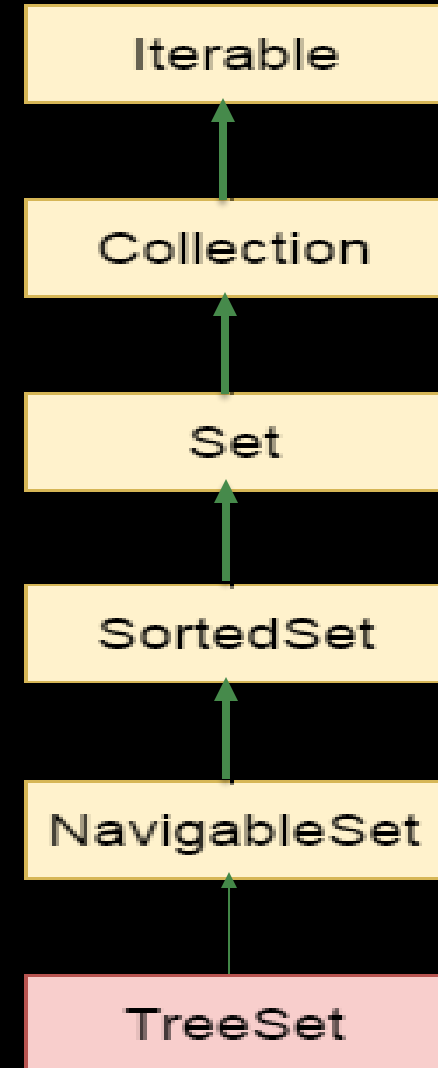
- Both LinkedHashSet and HashSet implements the Set interface. However, there exist some differences between them.
- LinkedHashSet maintains a linked list internally. Due to this, it maintains the insertion order of its elements.
- The LinkedHashSet class requires more storage than HashSet. This is because LinkedHashSet maintains linked lists internally.
- The performance of LinkedHashSet is slower than HashSet. It is because of linked lists present in LinkedHashSet.

Treeset

- TreeSet is similar to HashSet except that it sorts the elements in the ascending order while HashSet doesn't maintain any order.
- TreeSet doesn't allow null element like HashSet.
- TreeSet class implements the Set interface that uses a tree for storage.
- TreeSet implements the SortedSet interface so duplicate values are not allowed.
- TreeSet does not preserve the insertion order of elements but elements are sorted by keys.

Treeset Hierarchy

- Java TreeSet class implements NavigableSet interface.
- The NavigableSet interface extends SortedSet, Set, Collection and Iterable interfaces in hierarchical order.



TreeSet class declaration

- **public class TreeSet<E> extends AbstractSet<E> implements NavigableSet<E>, Cloneable, Serializable**

Constructor	Description
TreeSet()	It is used to construct an empty tree set that will be sorted in an ascending order according to the natural order of the tree set.
TreeSet(Collection c)	It is used to build a new tree set that contains the elements of the collection c.
TreeSet(Comparator comp)	It is used to construct an empty tree set that will be sorted according to given comparator.
TreeSet(SortedSet ss)	It is used to build a TreeSet that contains the elements of the given SortedSet.

Methods of Java TreeSet class

Method	Description
boolean addAll(Collection c)	It is used to add all of the elements in the specified collection to this set.
boolean contains(Object o)	It is used to return true if this set contains the specified element.
boolean isEmpty()	It is used to return true if this set contains no elements.
boolean remove(Object o)	It is used to remove the specified element from this set if it is present.
void add(Object o)	It is used to add the specified element to this set if it is not already present.
void clear()	It is used to remove all of the elements from this set.
Object clone()	It is used to return a shallow copy of this TreeSet instance.
Object first()	It is used to return the first (lowest) element currently in this sorted set.
Object last()	It is used to return the last (highest) element currently in this sorted set.
int size()	It is used to return the number of elements in this set.

Treeset Example

```
public class TreeSetExample {  
    public static void main(String args[]) {  
        TreeSet<String> tset = new TreeSet<String>();  
        tset.add("ABC");  
        tset.add("String");  
        tset.add("Test");  
        tset.add("Pen");  
        tset.add("Ink");  
        tset.add("Jack");  
        System.out.println(tset);  
    }  
}
```

[ABC, Ink, Jack, Pen, String, Test]
[0, 3, 7, 88, 101, 222]

```
// TreeSet of Integer Type  
TreeSet<Integer> tset2 = new TreeSet<Integer>();  
  
// Adding elements to TreeSet<Integer>  
tset2.add(88);  
tset2.add(7);  
tset2.add(101);  
tset2.add(0);  
tset2.add(3);  
tset2.add(222);  
System.out.println(tset2);  } }
```

Output: You can see both the TreeSet have been sorted in ascending order implicitly.

HashSet vs TreeSet

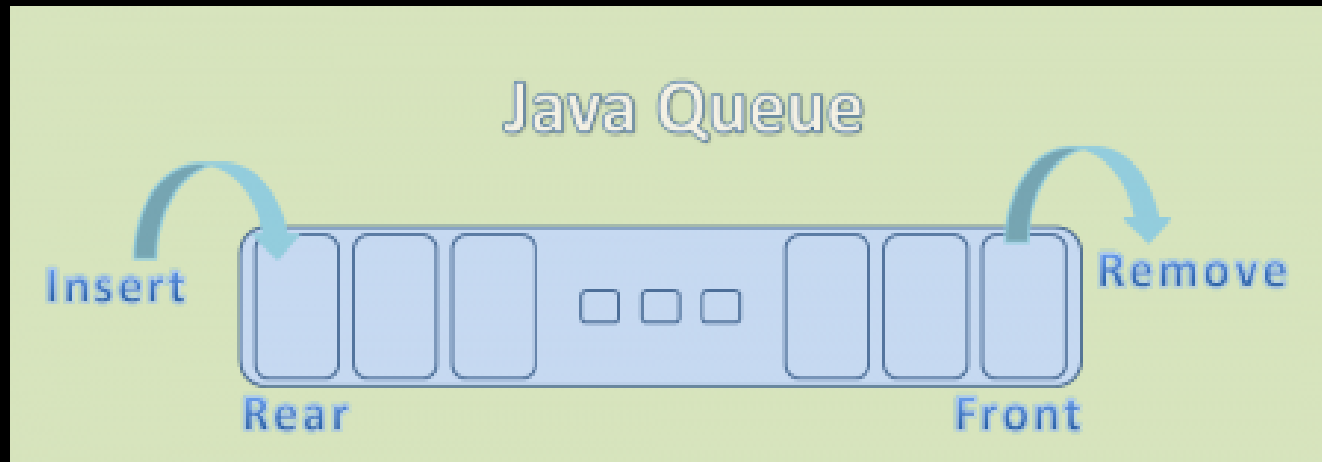
- HashSet gives better performance (faster) than TreeSet for the operations like add, remove, contains, size etc.
- HashSet offers constant time cost while TreeSet offers $\log(n)$ time cost for such operations.
- HashSet does not maintain any order of elements while TreeSet elements are sorted in ascending order by default.
- If you want a sorted Set then it is better to add elements to HashSet and then convert it into TreeSet rather than creating a TreeSet and adding elements to it.
- Both of these classes are non-synchronized that means they are not thread-safe and should be synchronized explicitly when there is a need of thread-safe operations.

Queue Interface

- Java Queue interface orders the element in FIFO(First In First Out) manner.
- In FIFO, first element is removed first and last element is removed at last.
- A Queue is designed in such a way so that the elements added to it are placed at the end of Queue and removed from the beginning of Queue.
- Java Queue is an interface available in `java.util` package and extends `java.util.Collection` interface.

Queue Interface declaration

```
public interface Queue<E> extends Collection<E>
```



Methods of Java Queue Interface

Method	Description
boolean add(object)	It is used to insert the specified element into this queue and return true upon success.
boolean offer(object)	It is used to insert the specified element into this queue.
Object remove()	It is used to retrieves and removes the head of this queue.
Object poll()	It is used to retrieves and removes the head of this queue, or returns null if this queue is empty.
Object element()	It is used to retrieves, but does not remove, the head of this queue.
Object peek()	It is used to retrieves, but does not remove, the head of this queue, or returns null if this queue is empty.

Queue Interface

- Since it is an interface, we need a concrete class during its declaration.
- There are many ways to initialize a Queue object, most common being-
- As a Priority Queue
- As a LinkedList

PriorityQueue class

- The PriorityQueue class provides the facility of using queue.
- But it does not orders the elements in FIFO manner.
- It inherits AbstractQueue class.

PriorityQueue class declaration

- **public class PriorityQueue<E> extends AbstractQueue<E> implements Serializable**

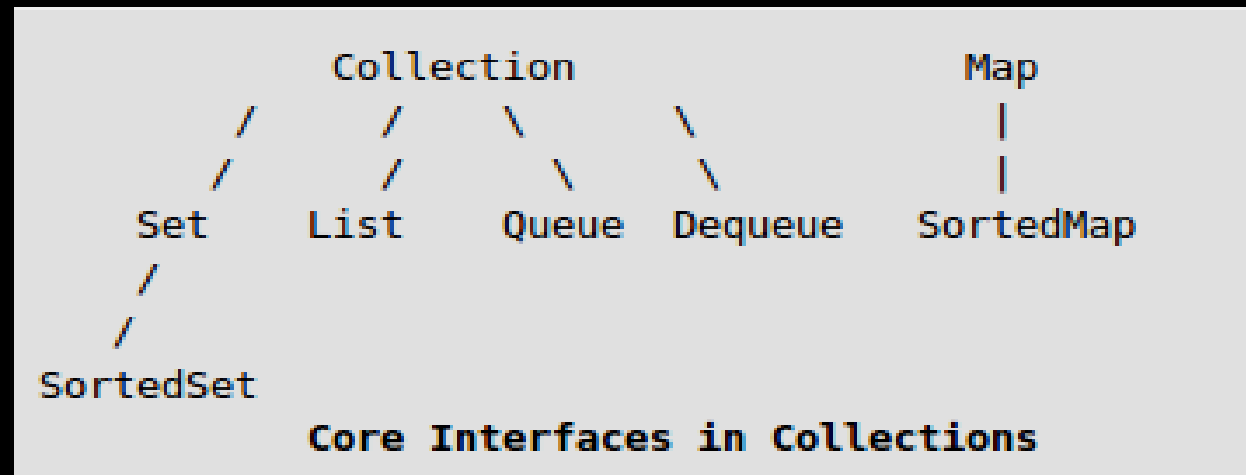
Queue Demo

```
class TestCollection{
public static void main(String args[]){
PriorityQueue<String> queue=new
PriorityQueue<String>();
queue.add("John");
queue.add("Jim");
queue.add("Mary");
queue.add("Je");
queue.add("Raul");
System.out.println("head:"+queue.element());
System.out.println("head:"+queue.peek());
System.out.println("iterating the queue
elements:");
Iterator itr=queue.iterator();
while(itr.hasNext()){
System.out.println(itr.next());
}
```

```
queue.remove();
queue.poll();
System.out.println("after removing two
elements:");
Iterator<String> itr2=queue.iterator();
while(itr2.hasNext()){
System.out.println(itr2.next()); } } }
```

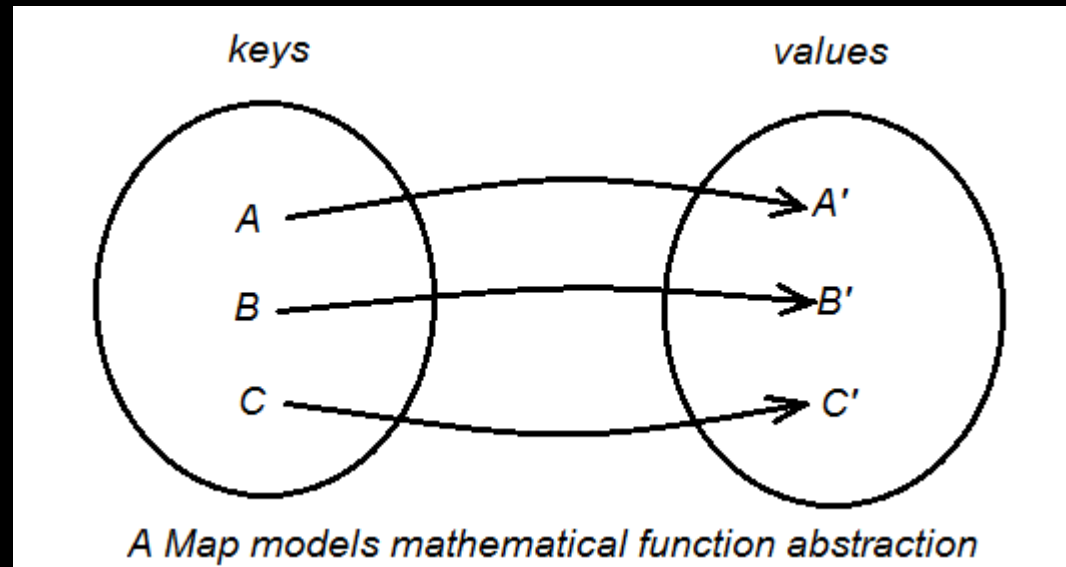
Map Interface

- The `java.util.Map` interface represents a mapping between a key and a value.
- The `Map` interface is not a subtype of the `Collection` interface.
- Therefore it behaves a bit different from the rest of the collection types.



Map Interface

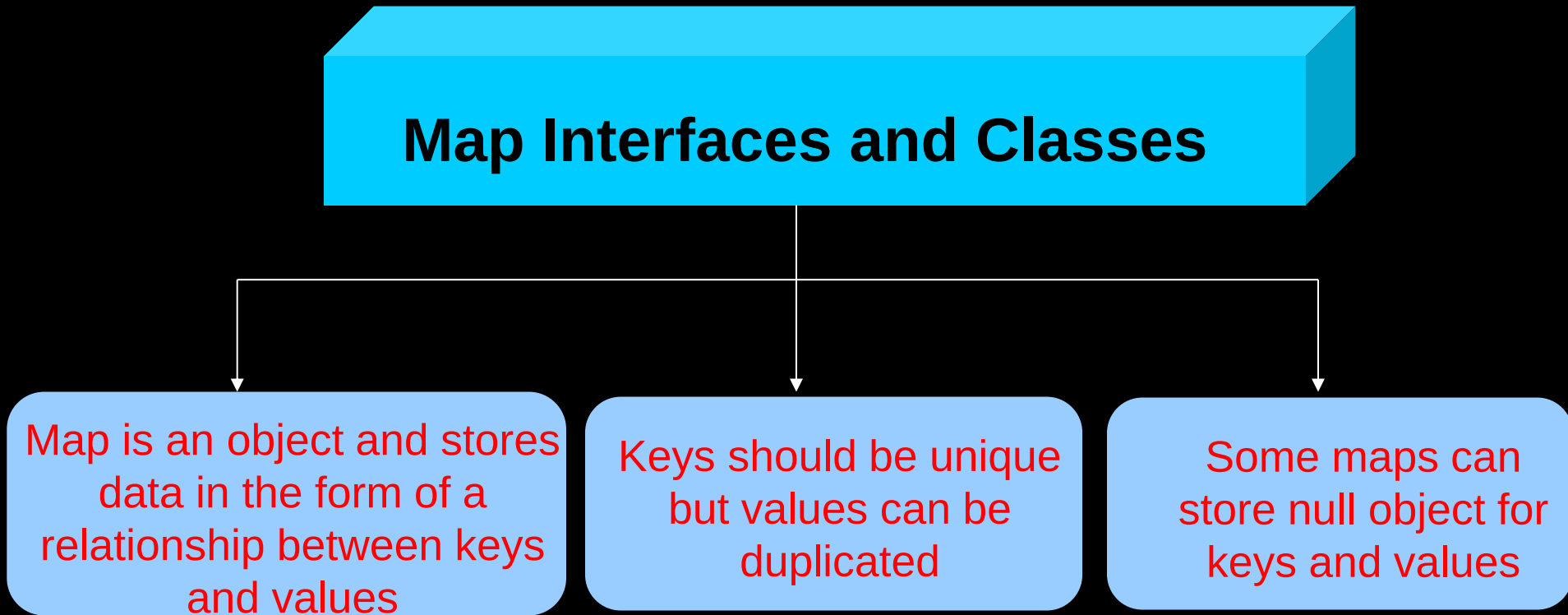
- A Map cannot contain duplicate keys and each key can map to at most one value.
- Some implementations allow null key and null value (HashMap and LinkedHashMap) but some do not (TreeMap).



Why and When Use Maps

- Maps are perfectly for key-value association mapping such as dictionaries.
- Use Maps when you want to retrieve and update elements by keys, or perform lookups by keys.
 - A map of error codes and their descriptions.
 - A map of zip codes and cities.
 - A map of managers and employees. Each manager (key) is associated with a list of employees (value) he manages.
 - A map of classes and students. Each class (key) is associated with a list of students (value).

Map Interfaces and Classes 4-1



Map -Demo

```
Map<String, String> mapCountryCodes = new HashMap<>();
    mapCountryCodes.put("1", "USA");
mapCountryCodes.put("44", "United Kingdom");
mapCountryCodes.put("33", "France");
mapCountryCodes.put("81", "Japan");
    Set<String> setCodes = mapCountryCodes.keySet();
Iterator<String> iterator = setCodes.iterator();

while (iterator.hasNext()) {
    String code = iterator.next();
    String country = mapCountryCodes.get(code);
    System.out.println(code + " => " + country);
}
```

Map Interfaces and Classes 4-2

- It maps keys to values .
- Each key can map to one value only.
- It contains methods for basic operations, bulk operations and collection views.
- `SortedMap` interface extends `Map` interface.
- `SortedMap` interface maintains its entries in ascending order.

Commonly used Methods defined by Map

- `boolean containsKey(Object k)`: returns true if map contain k as key. Otherwise false.
- `Object get(Object k)` : returns values associated with the key k.
- `Object put(Object k, Object v)` : stores an entry in map.
- `Object putAll(Map m)` : put all entries from m in this map.
- `Set keySet()` : returns Set that contains the key in a map.
- `Set entrySet()` : returns Set that contains the entries in a map.

Map.Entry Interface

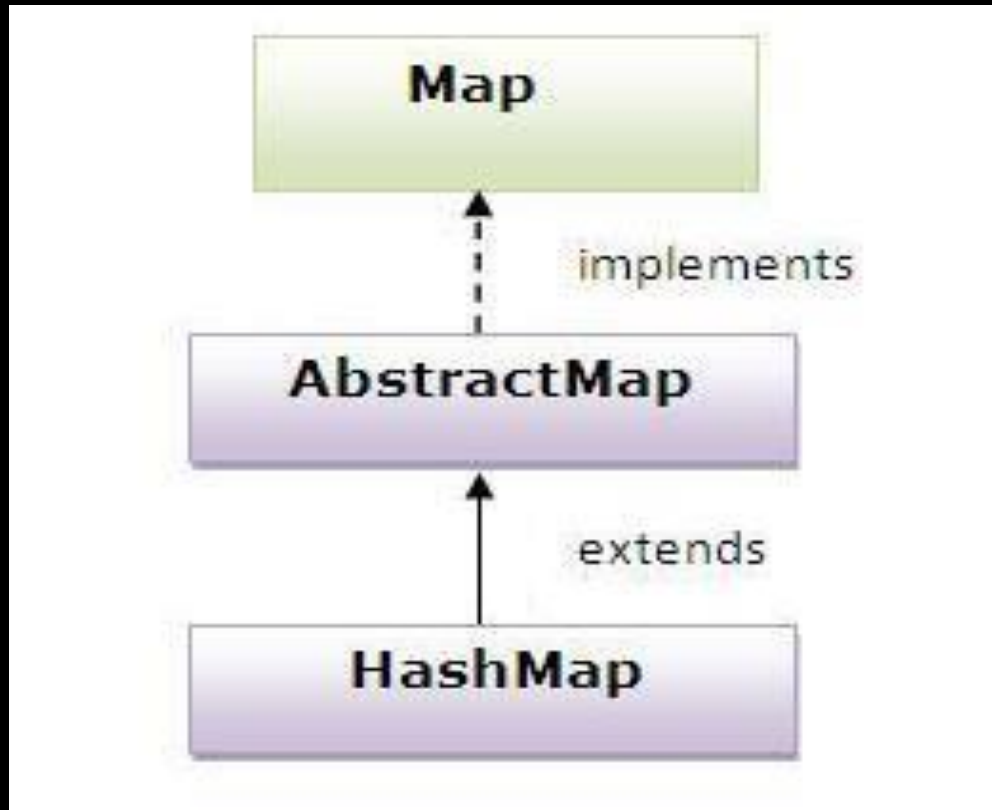
- Entry is the sub interface of Map.
- So we will be accessed it by Map.Entry name.
- It provides methods to get key and value.

Method	Description
Object getKey()	It is used to obtain key.
Object getValue()	It is used to obtain value.

Hash Map

- A HashMap contains values based on the key.
- It implements the Map interface and extends AbstractMap class.
- It contains only unique elements.
- It may have one null key and multiple null values.
- It maintains no order.

Hierarchy of HashMap class



HashMap 2-2

- The constructors of this class are:

Constructor	Description
HashMap()	Creates an empty map with default capacity and load factor.
HashMap(int size)	Creates an empty map with the capacity specified in size and default load factor.
HashMap(int size, float load)	Creates an empty map with the capacity specified in size and load factor specified by load.
HashMap (Map map)	Creates a hash map with mappings as the specified map

❑ It allows null values.

Demo Hash Map

```
import java.util.*;
class HashMapDemo{
public static void main(String args[]){

    HashMap<Integer,String> hm=new HashMap<Integer,String>();

    hm.put(111,"John");
    hm.put(222,"Jim"); hm.put(444,null);
    hm.put(333,"Mike"); hm.put(null,"Jimmy"); hm.put(null,"Jack");

    for(Map.Entry m:hm.entrySet()){
        System.out.println(m.getKey()+" "+m.getValue());
    }
}
```

Difference between HashMap and HashSet

HashMap	Hash Set
HashMap is an implementation of Map interface	HashSet is an implementation of Set Interface
HashMap Stores data in form of key-value pair	HashSet Store only objects
Put method is used to add element in map	Add method is used to add element in Set
In hash map hashCode value is calculated using key object	Here member object is used for calculating hashCode value which can be same for two objects so equal () method is used to check for equality if it returns false that means two objects are different.
HashMap is faster than HashSet because unique key is used to access object	HashSet is slower than Hashmap

LinkedHashMap

- The LinkedHashMap Class is just like HashMap with an additional feature of maintaining an order of elements inserted into it.
- HashMap provided the advantage of quick insertion, search, and deletion but it never maintained the track and order of insertion.
- The LinkedHashMap provides where the elements can be accessed in their insertion order.

Features of a LinkedHashMap

A LinkedHashMap contains values based on the key. It implements the Map interface and extends the HashMap class.

It contains only unique elements.

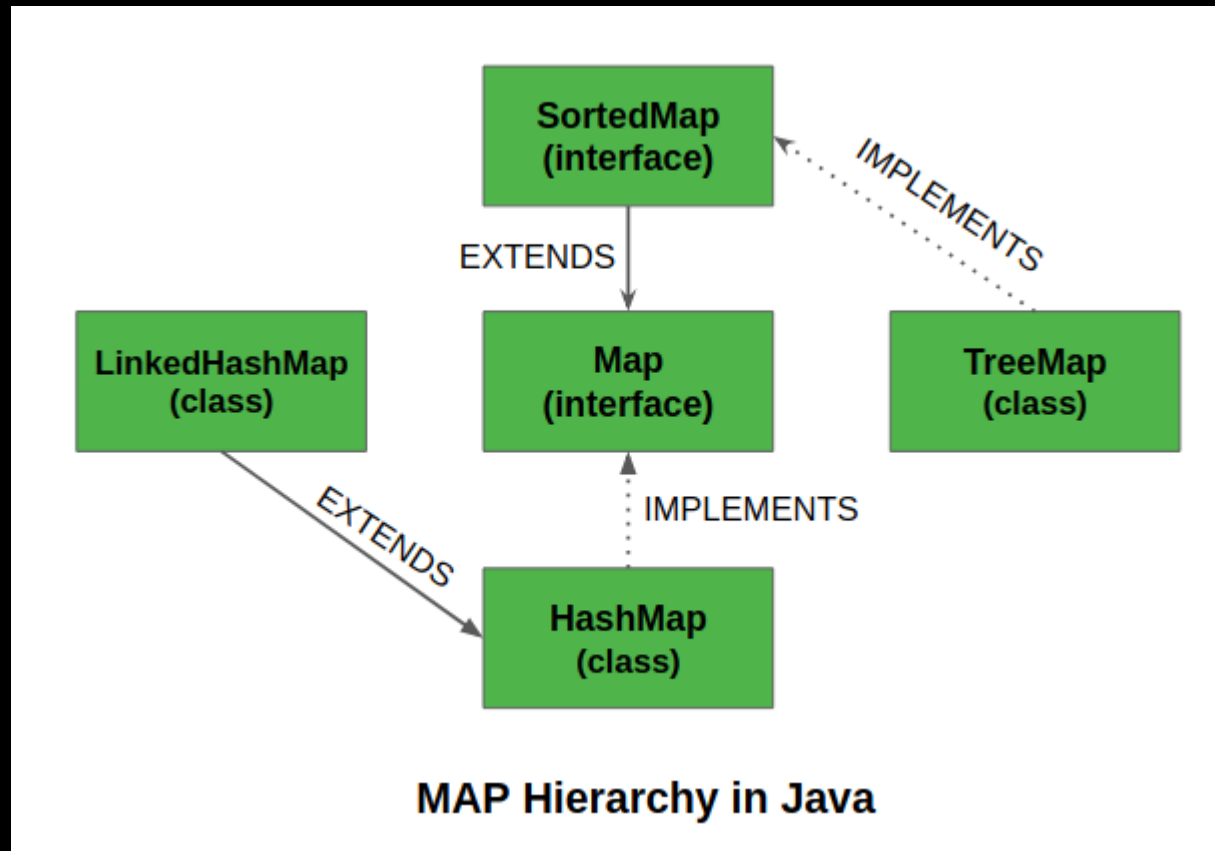
It may have one null key and multiple null values.

It is non-synchronized.

It is the same as HashMap with an additional feature that it maintains insertion order. For example, when we run the code with a HashMap, we get a different order of elements.

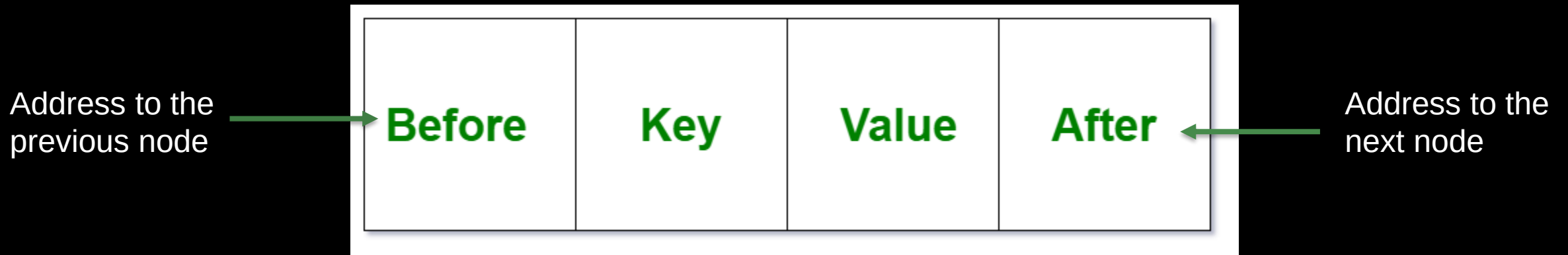
Declaration of LinkedHashMap

```
public class LinkedHashMap<K,V> extends HashMap<K,V> implements Map<K,V>
```



LinkedHashMap Structure

- The data is stored in the form of nodes.
- The implementation of the LinkedHashMap is very similar to a doubly-linked list.
- Therefore, each node of the LinkedHashMap is represented as:



Constructors of Java LinkedHashMap

Constructor	Description
<code>LinkedHashMap()</code>	It is used to construct a default LinkedHashMap.
<code>LinkedHashMap(int capacity)</code>	It is used to initialize a LinkedHashMap with the given capacity.
<code>LinkedHashMap(int capacity, float loadFactor)</code>	It is used to initialize both the capacity and the load factor.
<code>LinkedHashMap(int capacity, float loadFactor, boolean accessOrder)</code>	It is used to initialize both the capacity and the load factor with specified ordering mode.
<code>LinkedHashMap(Map<? extends K,? extends V> m)</code>	It is used to initialize the LinkedHashMap with the elements from the given Map class m.

Methods of LinkedHashMap

- `put()` - inserts the specified key/value mapping to the map
- `putAll()` - inserts all the entries from the specified map to this map
- `putIfAbsent()` - inserts the specified key/value mapping to the map if the specified key is not present in the map
- `entrySet()` - returns a set of all the key/value mapping of the map
- `keySet()` - returns a set of all the keys of the map
- `values()` - returns a set of all the values of the map
- `remove(key)` - returns and removes the entry associated with the specified key from the map
- `remove(key, value)` - removes the entry from the map only if the specified key mapped to be the specified value and return a boolean value

LinkedHashMap Demo

```
class Main {  
    public static void main(String[] args) {  
        LinkedHashMap<String, Integer> numbers = new LinkedHashMap<>();  
        numbers.put("One", 1);  
        numbers.put("Two", 2);  
        numbers.put("Three", 3);  
        System.out.println("LinkedHashMap: " + numbers);  
  
        // Using entrySet()  
        System.out.println("Key/Value mappings: " + numbers.entrySet());  
        // Using keySet()  
        System.out.println("Keys: " + numbers.keySet());  
        // Using values()  
        System.out.println("Values: " + numbers.values()); } }
```

Output

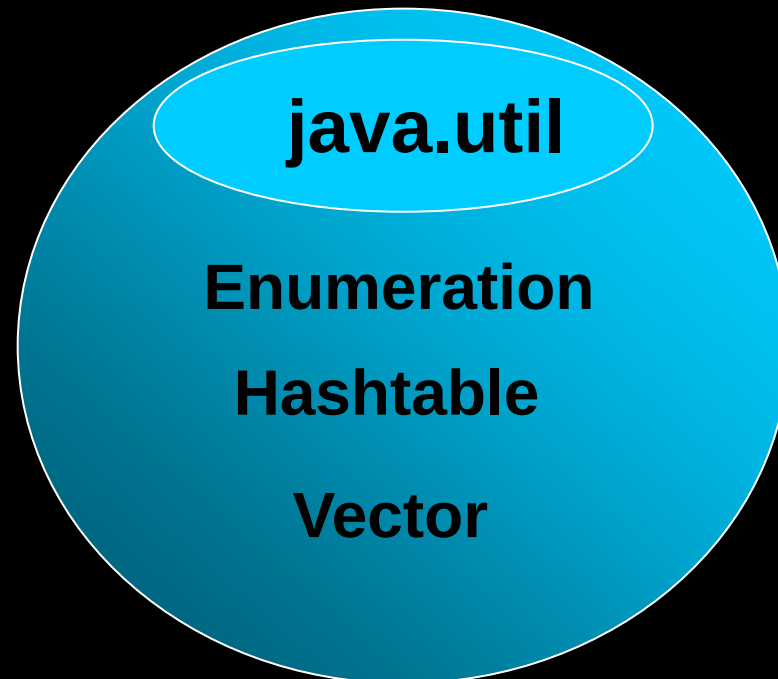
LinkedHashMap: {One=1, Two=2, Three=3}
Key/Value mappings: [One=1, Two=2, Three=3]
Keys: [One, Two, Three]
Values: [1, 2, 3]

LinkedHashMap Vs. HashMap

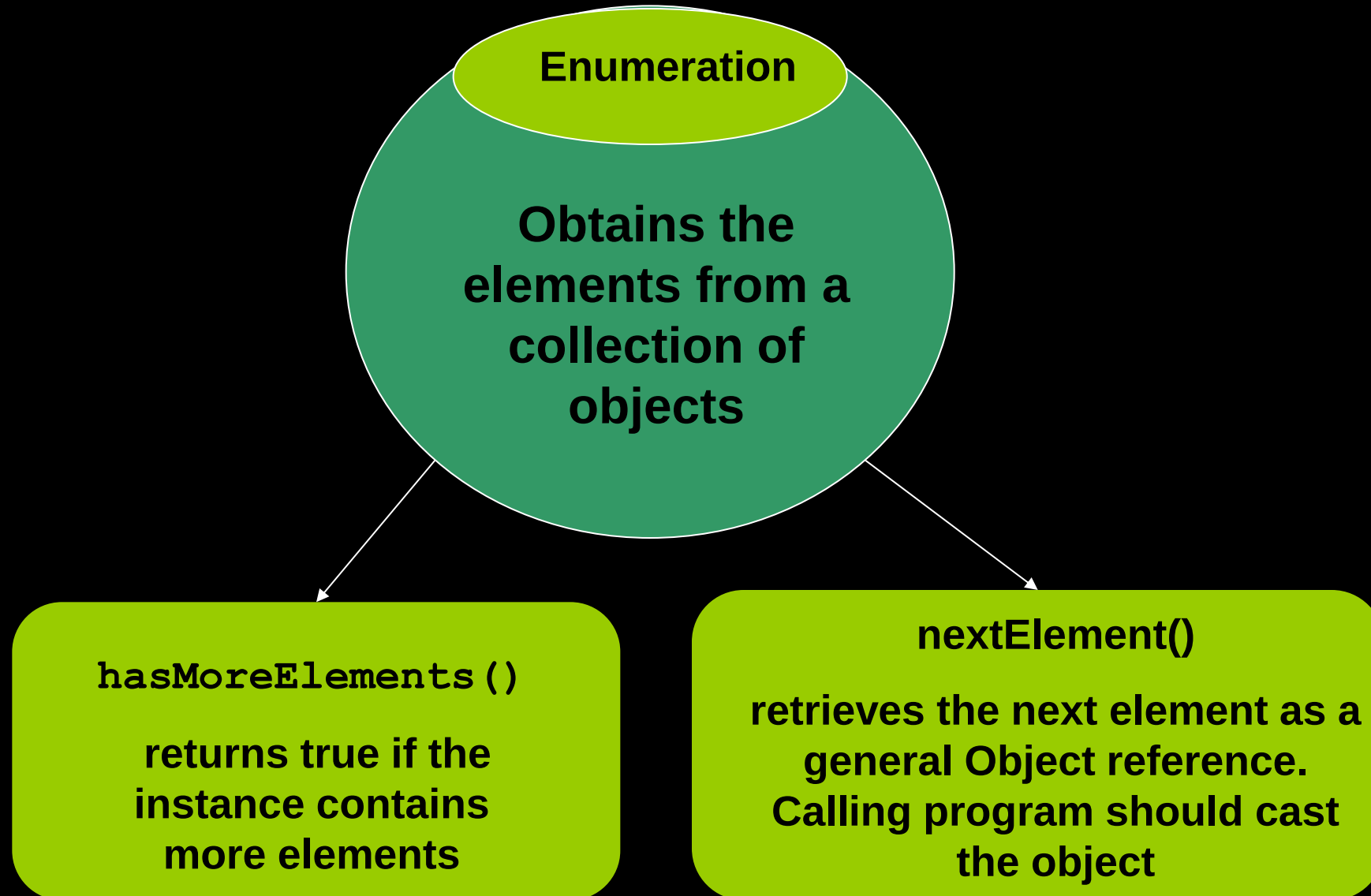
- Both the LinkedHashMap and the HashMap implements the Map interface. However, there exist some differences between them.
- LinkedHashMap maintains a doubly-linked list internally. Due to this, it maintains the insertion order of its elements.
- The LinkedHashMap class requires more storage than HashMap. This is because LinkedHashMap maintains linked lists internally.
- The performance of LinkedHashMap is slower than HashMap.

Legacy classes and Interfaces

- Earlier version of Java contained some classes and interfaces to store objects.
- Legacy classes are synchronized as opposed to the classes in the collection framework.



Enumeration



Hashtable class 2-1

- A Hashtable is an array of list.
- Each list is known as a bucket.
- The position of bucket is identified by calling the `hashCode()` method.
- A Hashtable contains values based on the key.
- It implements the Map interface and extends Dictionary class.
- It contains only unique elements.
- It may not have any null key or value.
- It is synchronized.

Hashtable class 2-2

```
StudentHashTest() {  
    hashtableStudents = new Hashtable();  
}
```

1

IS used to convert the
key to its equivalent

```
void add() {  
    hashtableStudents.put("Tony" , n  
    hashtableStudents.put("Cathy" ,  
    hashtableStudents.put("Michael"  
    hashtableStudents.put("Priscilla  
    hashtableStudents.put("Mark" , n  
}
```

```
void display() {  
    String name = "Michael";  
    System.out.println("*****"  
    + "*****");  
    System.out.println("Retreiving a Single Student Information");  
    System.out.println("*****"  
    + "*****");  
    System.out.println("\nStudent name is: " + name);  
    String re = (String) hashtableStudents.get(name);  
    System.out.println("Student completed in the Year: " + result);  
    System.out.println("*****");  
    System.out.println("Details");  
    System.out.println("*****\n");  
    Enumeration enumNames = hashtableStudents.keys();  
    while (enumNames.hasMoreElements()) {  
        name = (String) enumNames.nextElement();  
        System.out.println(name + " completed graduation in "  
        + hashtableStudents.get(name) + "\n");  
    }  
}
```

Displaying a single
element's value

Values are added to
Hashtable objects by using
the put() method

Size as
Added.

Iterating through the entire Hashtable and displaying
the value using Enumeration interface

Demonstration: Example 2

Difference between HashMap and Hashtable

HashMap	Hashtable
1) HashMap is non synchronized . It is not-thread safe and can't be shared between many threads without proper synchronization code.	Hashtable is synchronized . It is thread-safe and can be shared with many threads.
2) HashMap allows one null key and multiple null values .	Hashtable doesn't allow any null key or value .
3) HashMap is a new class introduced in JDK 1.2 .	Hashtable is a legacy class .
4) HashMap is fast .	Hashtable is slow .
6) HashMap is traversed by Iterator .	Hashtable is traversed by Enumerator and Iterator .
8) HashMap inherits AbstractMap class.	Hashtable inherits Dictionary class.

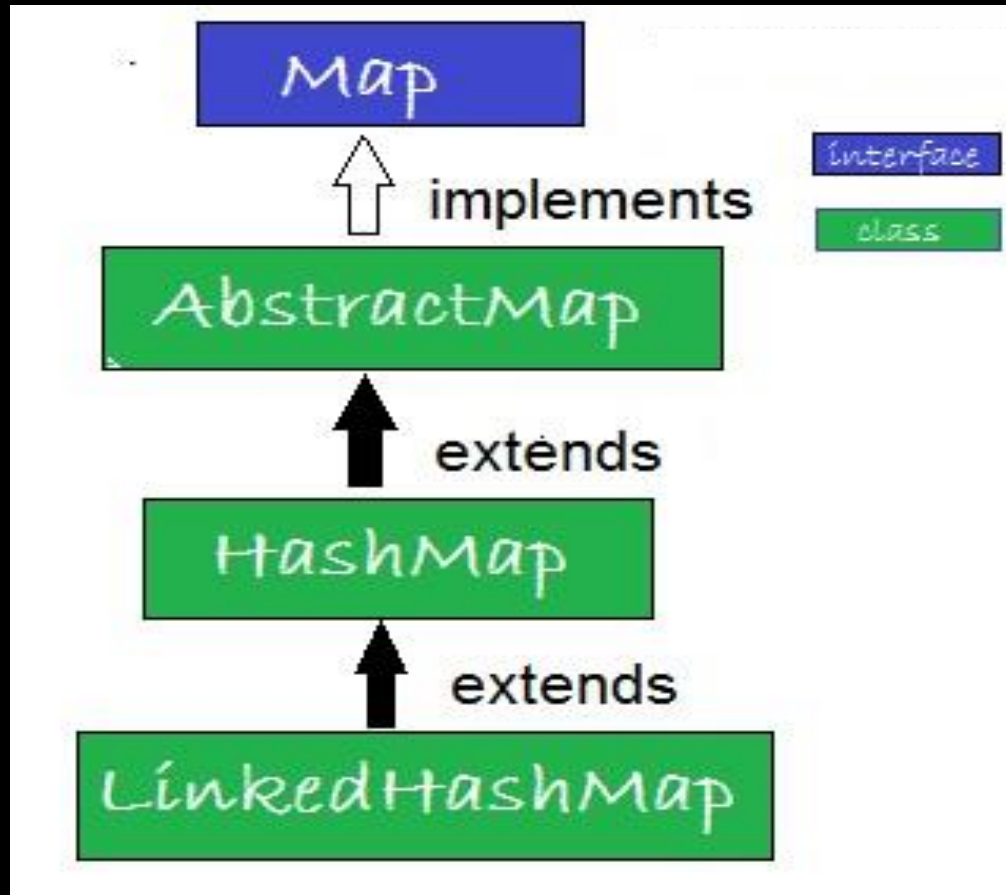
LinkedHashMap Class

- **LinkedHashMap** extends **HashMap** class.
- It maintains a linked list of entries in map in order in which they are inserted.
- **LinkedHashMap** is a Hash table and linked list implementation of the Map interface, with predictable iteration order.
- This implementation differs from **HashMap** in that it maintains a doubly-linked list running through all of its entries.
- This linked list defines the iteration ordering, which is normally the order in which keys were inserted into the map (insertion-order).

LinkedHashMap vs Hash & TreeMap

- **HashMap doesn't maintain any order.**
- **TreeMap sort the entries in ascending order of keys.**
- **LinkedHashMap maintains the insertion order.**
- **LinkedHashMap is like HashMap with additional feature that we can access elements in their insertion order.**

Hierarchy of LinkedHashMap Class



Constructors

Constructor	Description
<code>LinkedHashMap()</code>	It is used to construct a default <code>LinkedHashMap</code> .
<code>LinkedHashMap(int capacity)</code>	It is used to initialize a <code>LinkedHashMap</code> with the given capacity.
<code>LinkedHashMap(int capacity, float fillRatio)</code>	It is used to initialize both the capacity and the fillRatio.
<code>LinkedHashMap(Map m)</code>	It is used to initialize the <code>LinkedHashMap</code> with the elements from the given <code>Map</code> class <code>m</code> .

Methods

Apart from the methods inherited from its parent classes, **LinkedHashMap** defines the following methods –

Method	Description
Object get(Object key)	It is used to return the value to which this map maps the specified key.
void clear()	It is used to remove all mappings from this map.
boolean containsKey(Object key)	It is used to return true if this map maps one or more keys to the specified value.

LinkedHashMap Demo

```
public class LinkedHashMapDemo {  
    public static void main(String args[]) {  
        // HashMap Declaration  
        LinkedHashMap<Integer, String> lhmap =  
            new LinkedHashMap<Integer, String>();  
  
        //Adding elements to LinkedHashMap  
        lhmap.put(22, "Abey");  
        lhmap.put(33, "Dawn");  
        lhmap.put(1, "Sherry");  
        lhmap.put(2, "Karon");  
        lhmap.put(100, "Jim");  
    }  
}
```

```
        // remove value  
        lhmap.remove(33);  
  
        // Generating a Set of entries  
        Set set = lhmap.entrySet();  
  
        // Displaying elements of LinkedHashMap  
        Iterator iterator = set.iterator();  
        while(iterator.hasNext()) {  
            Map.Entry me = (Map.Entry)iterator.next();  
            System.out.print("Key is: "+ me.getKey() +  
                "& Value is: "+me.getValue()+"\n");  
        }  
    }  
}
```

Values are returned in the same order in which they got inserted.

TreeMap Class

- Java TreeMap class implements the Map interface by using a tree.
- It provides an efficient means of storing key/value pairs in sorted order.
- A TreeMap contains values based on the key.
- It implements the NavigableMap interface and extends AbstractMap class.
- It contains only unique elements.
- It cannot have null key but can have multiple null values.
- It is same as HashMap instead maintains ascending order.

TreeMap class declaration

- **public class TreeMap<K,V> extends AbstractMap<K,V> implements NavigableMap<K,V>, Cloneable, Serializable**

Constructor	Description
TreeMap()	It is used to construct an empty tree map that will be sorted using the natural order of its key.
TreeMap(Comparator comp)	It is used to construct an empty tree-based map that will be sorted using the comparator comp.
TreeMap(Map m)	It is used to initialize a tree map with the entries from m , which will be sorted using the natural order of the keys.
TreeMap(SortedMap sm)	It is used to initialize a tree map with the entries from the SortedMap sm , which will be sorted in the same order as sm .

Methods of Java TreeMap class

Method	Description
boolean containsKey(Object key)	It is used to return true if this map contains a mapping for the specified key.
boolean containsValue(Object value)	It is used to return true if this map maps one or more keys to the specified value.
Object firstKey()	It is used to return the first (lowest) key currently in this sorted map.
Object get(Object key)	It is used to return the value to which this map maps the specified key.
Object lastKey()	It is used to return the last (highest) key currently in this sorted map.
Object remove(Object key)	It is used to remove the mapping for this key from this TreeMap if present.
void putAll(Map map)	It is used to copy all of the mappings from the specified map to this map.
Set entrySet()	It is used to return a set view of the mappings contained in this map.
int size()	It is used to return the number of key-value mappings in this map.
Collection values()	It is used to return a collection view of the values contained in this map.

TreeMap Demo

```
public class Details {  
    public static void main(String args[]) {  
        TreeMap<Integer, String> tmap =  
            new TreeMap<Integer, String>();  
  
        /*Adding elements to TreeMap*/  
        tmap.put(1, "Data1");  
        tmap.put(23, "Data2");  
        tmap.put(70, "Data3");  
        tmap.put(4, "Data4");  
        tmap.put(2, "Data5");
```

```
        /* Display content using Iterator*/  
        Set set = tmap.entrySet();  
        Iterator iterator = set.iterator();  
        while(iterator.hasNext()) {  
            Map.Entry mentry =  
                (Map.Entry)iterator.next();  
            System.out.print("key is: "+  
                mentry.getKey() + " & Value is: ");  
            System.out.println(mentry.getValue());  
        } } }
```

Inserted data is in random order however when we displayed the TreeMap content we get the sorted result in the ascending order of keys.

HashMap Vs TreeMap

BASIS FOR COMPARISON	HASHMAP	TREEMAP
Basic	HashMap does not maintain insertion order.	TreeMap maintains insertion order.
DataStructure	HashMap uses Hash Table as an underlying data structure.	TreeMap uses Red-Black Tree as an underlying data structure.
Null Keys and Values	HashMap allows Null key once and Null value any number of times.	TreeMap does not allow Null key but allows Null Values any number of times.
Extends and Implements	HashMap extends AbstractMap class and implements Map interface.	TreeMap extends AbstractMap class and implements SortedMap and NavigableMap interface.
Performance	HashMap operates faster.	TreeMap in comparison to HashMap operates slower.

Method of Collections class

- **java.util.Collections.sort()**
 - method is present in java.util.Collections class.
 - It is used to sort the elements present in the specified [list](#) of Collection in ascending order.
- **static int binarySearch(List list1, Object obj1):**
 - Searches the obj1 in the list list1. Returns the index number of the element obj1. Before applying this method, the elements must be sorted earlier with sort() method;

Method of Collections class

- **static void reverse(List list1):**
 - Existing order of the elements in the list list1 are reversed. Again reversing gets the original order.
- **static void swap(List list1, int index1, int index2):**
 - List list1 elements at index numbers index1 and index2 are swapped.
- **static void copy(List destination1, List source1):**
 - Copies all the elements of List source1 into the destination1 list. It is like [arraycopy\(\)](#) method.

Comparable Interface

- Java Comparable interface is used to order the objects of user-defined class.
- A comparable object is capable of comparing itself with another object.
- This interface is found in java.lang package and contains only one method named compareTo(Object).
- It provide single sorting sequence only i.e. you can sort the elements on based on single data member only.
- For example it may be rollno, name, age or anything else.
- Collections class only provides static methods for sorting the elements of a collection if it is of primitive types or String.

What is the Comparable Interface Used For?

- It is used to Compare and Sort Objects.



- Sorting Apples based on weight.
- When we are sorting them, we need to repeatedly compare two apples weights until all the apples are in the correct order.
- Comparable can't sort the objects on its own, but the interface defines a method `int compareTo(T)`.

How compareTo() Works



- The compareTo() method works by returning an int value that is either positive, negative, or zero.
- It compares the object by making the call to the object that is the argument.
- A negative number means that the object making the call is “less” than the argument.
- If we were comparing the apples by size, the above call would return a negative number, say -400, because the red apple is smaller than the green apple.
- If the two apples were of equal weight, the call would return 0.
- If the red apple was heavier, compareTo() would return a positive number, say 68.

Java Comparable Example

```
class Student implements
Comparable<Student>

{
int rollno;
String name;
int age;
Student(int rollno,String name,int age){
this.rollno=rollno;
this.name=name;
this.age=age;
}
```

```
public int compareTo(Student st)
{
if(age==st.age)
return 0;
else if(age>st.age)
return 1;
else
return -1;
}
}
```

Java Comparable Example

```
public class ComparableDemo{  
    public static void main(String args[]){  
        ArrayList<Student> al=new ArrayList<Student>();  
        al.add(new Student(111,"John",29));  
        al.add(new Student(222,"Mike",21));  
        al.add(new Student(555,"Mary",26));  
  
        Collections.sort(al);  
        for(Student st:al){  
            System.out.println(st.rollno+" "+st.name+" "+st.age);  
        } } }
```

Comparator Interface

- **Java Comparator interface** is used to order the objects of user-defined class.
- A comparator object is capable of comparing two objects of two different classes.
- This interface is found in java.util package and contains 2 methods **compare(Object obj1, Object obj2)** and **equals(Object element)**.
- It provides multiple sorting sequence i.e. you can sort the elements on the basis of any data member, for example rollno, name, age or anything else.

Methods of Java Comparator Interface

Method	Description
<code>public int compare(Object obj1, Object obj2)</code>	It compares the first object with the second object.
<code>public boolean equals(Object obj)</code>	It is used to compare the current object with the specified object.

How to Use Comparator

- Method of Collections class for sorting List elements is used to sort the elements of List by the given comparator.
- **ComparatorClass** must implement **Comparator** interface.
- `public void sort(List list, ComparatorClass c)`

Comparable	Comparator
1) Comparable provides a single sorting sequence . In other words, we can sort the collection on the basis of a single element such as id, name, and price.	The Comparator provides multiple sorting sequences . In other words, we can sort the collection on the basis of multiple elements such as id, name, and price etc.
2) Comparable affects the original class , i.e., the actual class is modified.	Comparator doesn't affect the original class , i.e., the actual class is not modified.
3) Comparable provides compareTo() method to sort elements.	Comparator provides compare() method to sort elements.
4) Comparable is present in java.lang package.	A Comparator is present in the java.util package.
5) We can sort the list elements of Comparable type by Collections.sort(List) method.	We can sort the list elements of Comparator type by Collections.sort(List, Comparator) method.

Properties Class

- The properties object contains key and value pair both as a string.
- The `java.util.Properties` class is the subclass of `Hashtable`.
- It can be used to get property value based on the property key.
- The `Properties` class provides methods to get data from properties file and store data into properties file.
- It can be used to get properties of system.

Example of Properties class to get all the system properties

```
import java.util.*;
import java.io.*;

public class Test {

    public static void main(String[] args) throws Exception{

        Properties p=System.getProperties();

        Set set=p.entrySet();

        Iterator itr=set.iterator();

        while(itr.hasNext()){

            Map.Entry entry=(Map.Entry)itr.next();

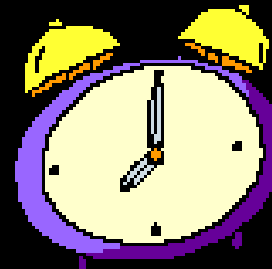
            System.out.println(entry.getKey()+" = "+entry.getValue()); } } }
```


Methods of Properties class

Method	Description
public void load(Reader r)	loads data from the Reader object.
public void load(InputStream is)	loads data from the InputStream object
public String getProperty(String key)	returns value based on the key.
public void setProperty(String key,String value)	sets the property in the properties object.
public void store(Writer w, String comment)	writers the properties in the writer object.
public void store(OutputStream os, String comment)	writes the properties in the OutputStream object.
storeToXML(OutputStream os, String comment)	writers the properties in the writer object for generating xml document.
public void storeToXML(Writer w, String comment, String encoding)	writers the properties in the writer object for generating xml document with specified encoding.

Java.util.Date

- Date class represents date and time.
- Provides methods for manipulating date and time components.
- One of the best applications of the Date class is in the creation of a real time clock.
- It provides constructors and methods to deal with date and time in java.



Date class constructors

```
void display() {  
    String strDate , strTime = "";  
    System.out.println("Today's date is : " +  
objDate);  
    long time = objDate.getTime();  
    System.out.println("Time in milliseconds  
+ " Jan 1,1970 (GMT): " + time);  
    strDate = objDate.toString();  
    // Extract GMT time  
    strTime = strDate.substring(11 ,  
(strDate.length() - 4));  
    // Extract the time in hours, minutes, seconds  
    strTime = "Time : " + strTime.substring(0 , 8);  
    System.out.println(strTime);  
}
```

```
DateTimeDisplay() {  
    objDate = new Date();  
}
```

Date object is used
to print the Date

Time is obtained from
the Date object by
using the getTime()
method

Demonstration: Example 1

java.util.Date Methods

Method	Description
boolean after(Date date)	tests if current date is after the given date.
boolean before(Date date)	tests if current date is before the given date.
Object clone()	returns the clone object of current date.
int compareTo(Date date)	compares current date with given date.
boolean equals(Date date)	compares current date with given date for equality.
static Date from(Instant instant)	returns an instance of Date object from Instant date.
long getTime()	returns the time represented by this date object.
int hashCode()	returns the hash code value for this date object.
void setTime(long time)	changes the current date and time to given time.
Instant toInstant()	converts current date into Instant object.
String toString()	converts this date into Instant object.

Java Calendar Class

- Java Calendar class is an abstract class.
- It provides methods for converting date between a specific instant in time and a set of calendar fields such as MONTH, YEAR, HOUR, etc.

Calendar Class

```
CalendarComponents() {  
    objCalendar = Calendar.getInstance();  
}  
  
void display() {  
    // Display the Date and Time components  
    System.out.println("\nDate and Time components:");  
    System.out.println("Month : " + objCalendar.get(Calendar.MONTH));  
    System.out.println("Day : " + objCalendar.get(Calendar.DATE));  
    System.out.println("Year : " + objCalendar.get(Calendar.YEAR));  
    System.out.println("Hour : " + objCalendar.get(Calendar.HOUR));  
    System.out.println("Minute : " + objCalendar.get(Calendar.MINUTE));  
    System.out.println("Second : " + objCalendar.get(Calendar.SECOND));  
    // Add 30 minutes to current time,  
    // then display date and time  
    objCalendar.add(Calendar.MINUTE , 30);  
    Date objDate = objCalendar.getTime();  
    System.out.println("\nDate and Time after adding 30" + "minutes to  
current time:\n");  
    System.out.println(objDate);  
}
```

An instance of Calendar

Calendar class defines certain integer constants that are used to get or set the components of Calendar

Demonstration: Example

Methods of Java Calendar

Method	Description
<code>abstract void add(int field, int amount)</code>	It is used to add or subtract the specified amount of time to the given calendar field, based on the calendar's rules.
<code>int get(int field)</code>	It is used to return the value of the given calendar field.
<code>static Calendar getInstance()</code>	It is used to get a calendar using the default time zone and locale.
<code>abstract int getMaximum(int field)</code>	It is used to return the maximum value for the given calendar field of this Calendar instance.
<code>abstract int getMinimum(int field)</code>	It is used to return the minimum value for the given calendar field of this Calendar instance.
<code>void set(int field, int value)</code>	It is used to set the given calendar field to the given value.
<code>void setTime(Date date)</code>	It is used to set this Calendar's time with the given Date.
<code>Date getTime()</code>	It is used to return a Date object representing this Calendar's time value.

Java.util.Locale Class

- ❑ The java.util.Locale class object represents a specific geographical, political, or cultural region. .Following are the important points about Locale –
 - ❑ An operation that requires a Locale to perform its task is called locale-sensitive and uses the Locale to form information for the user.
 - ❑ Locale is a mechanism for identifying objects, not a container for the objects themselves.

Constructors

- ❑ **Locale(String L)** : Creates Locale from the given language code.
- ❑ **Locale(String L, String C)** : Creates Locale from the given language, country code.
- ❑ **Locale(String L, String C, String V)** : Creates Locale from the given language, country, variant code.

Locale- Methods

- ❑ **getDisplayCountry()** : return the country to which locale belongs to.
- ❑ **getDefault()** : return the current – default value for the locale as per the JVM instance.
- ❑ **getCountry()** : return the country for the locale which could be empty or ISO 3166 2-letter code.
- ❑ **clone()** : creates clone of the locale.
- ❑ **getAvailableLocales()** : returns array of all the installed locales.
- ❑ **getDisplayLanguage()** : returns the language with the locale.
- ❑ **getDisplayName()** : displays name of the Locale

ResourceBundle class in Java

- ❑ The **ResourceBundle class** is used to internationalize the messages.
- ❑ It provides a mechanism to globalize the messages.
- ❑ The hardcoded message is not considered good in terms of programming, because it differs from one country to another.
- ❑ So we use the ResourceBundle class to globalize the messages.
- ❑ The ResourceBundle class loads these informations from the properties file that contains the messages.